

Chapter 12:

Image-Based Flow Visualization

Prof. Dr.-Ing. Thomas Schultz

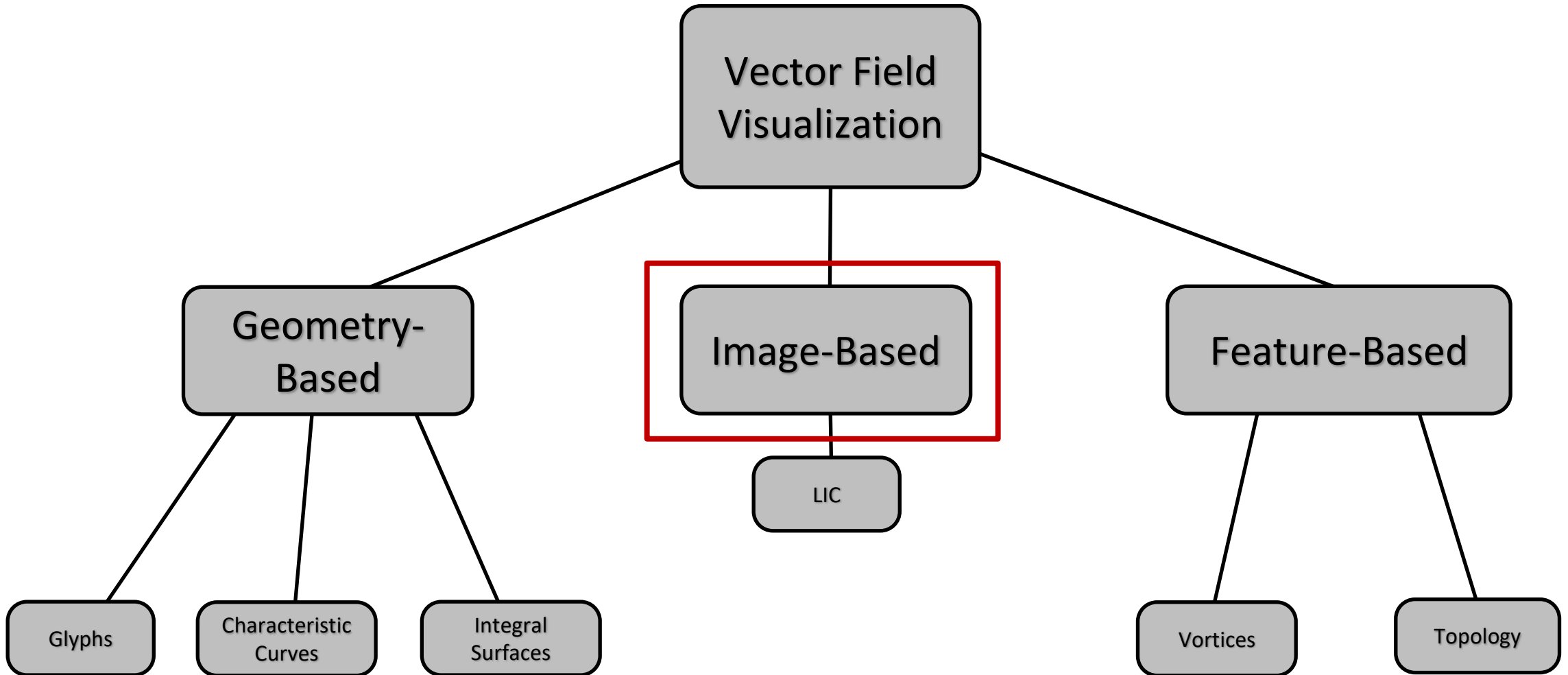
URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

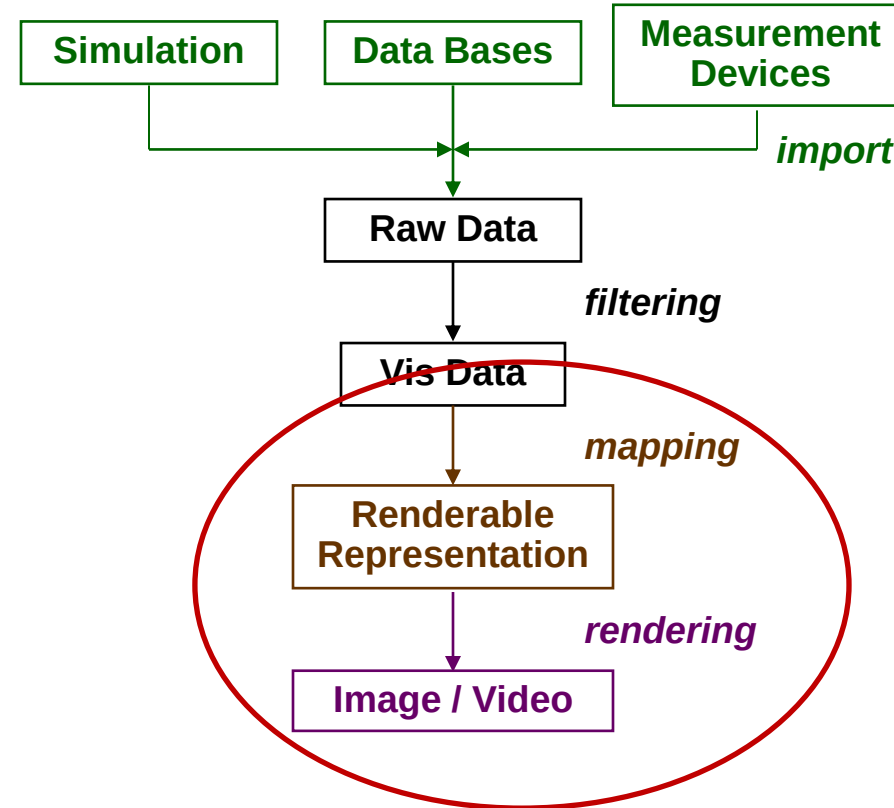
Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

June 30 and July 1, 2026

Overview of Vector Field Visualization



Where We Are In The Vis Pipeline

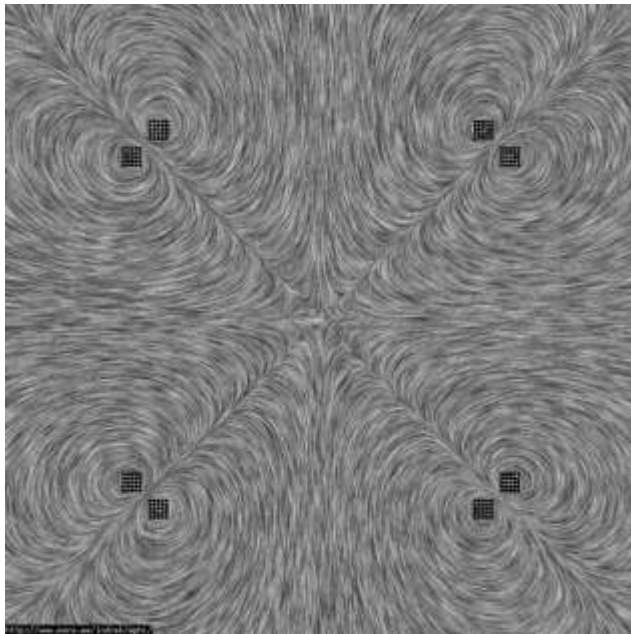


Section 12.1:

Line Integral Convolution (LIC)

Line Integral Convolution

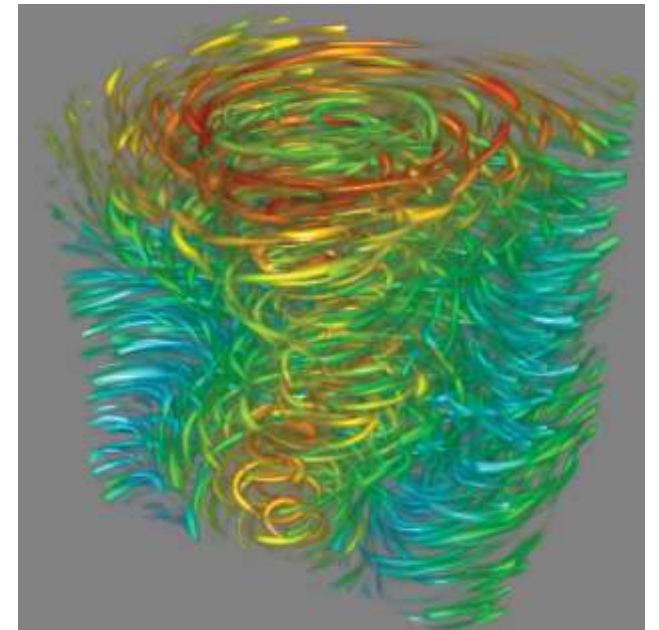
- **Line Integral Convolution (LIC)** [Cabral/Leedom, Siggraph 1993] is a dense image-based method to visualize vector fields



2D vector field



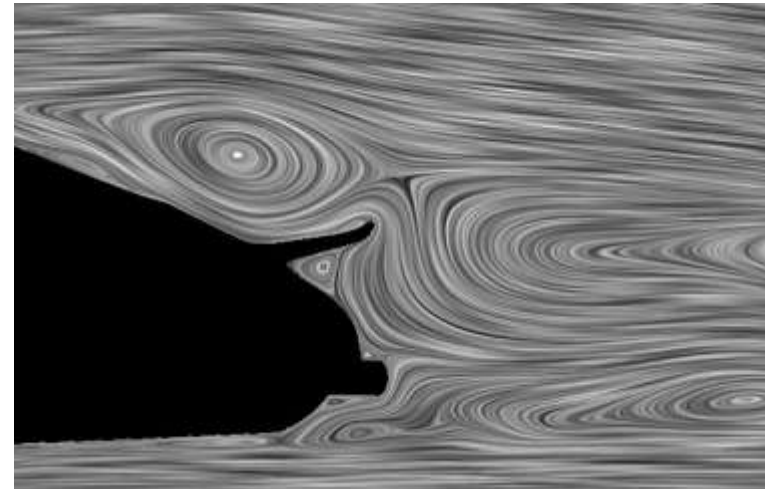
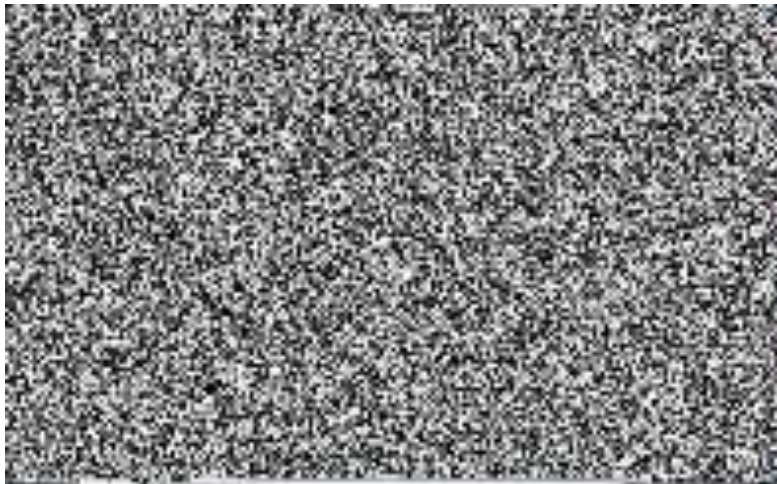
vector field on surface
(often called 2.5D)



3D vector field

Idea of Line Integral Convolution

- Image synthesis in LIC transforms a **random texture** into streaks that follow the local flow
- **Smears out** the texture along the streamlines in a vector field
 - Results in
 - **low correlation** of intensity values **between** neighboring lines,
 - but **high correlation along** them



Idea: Algorithm for Line Integral Convolution

Line Integral Convolution

- **Extracts** stream lines from the given vector field
- **Convolves** a random texture along the stream lines

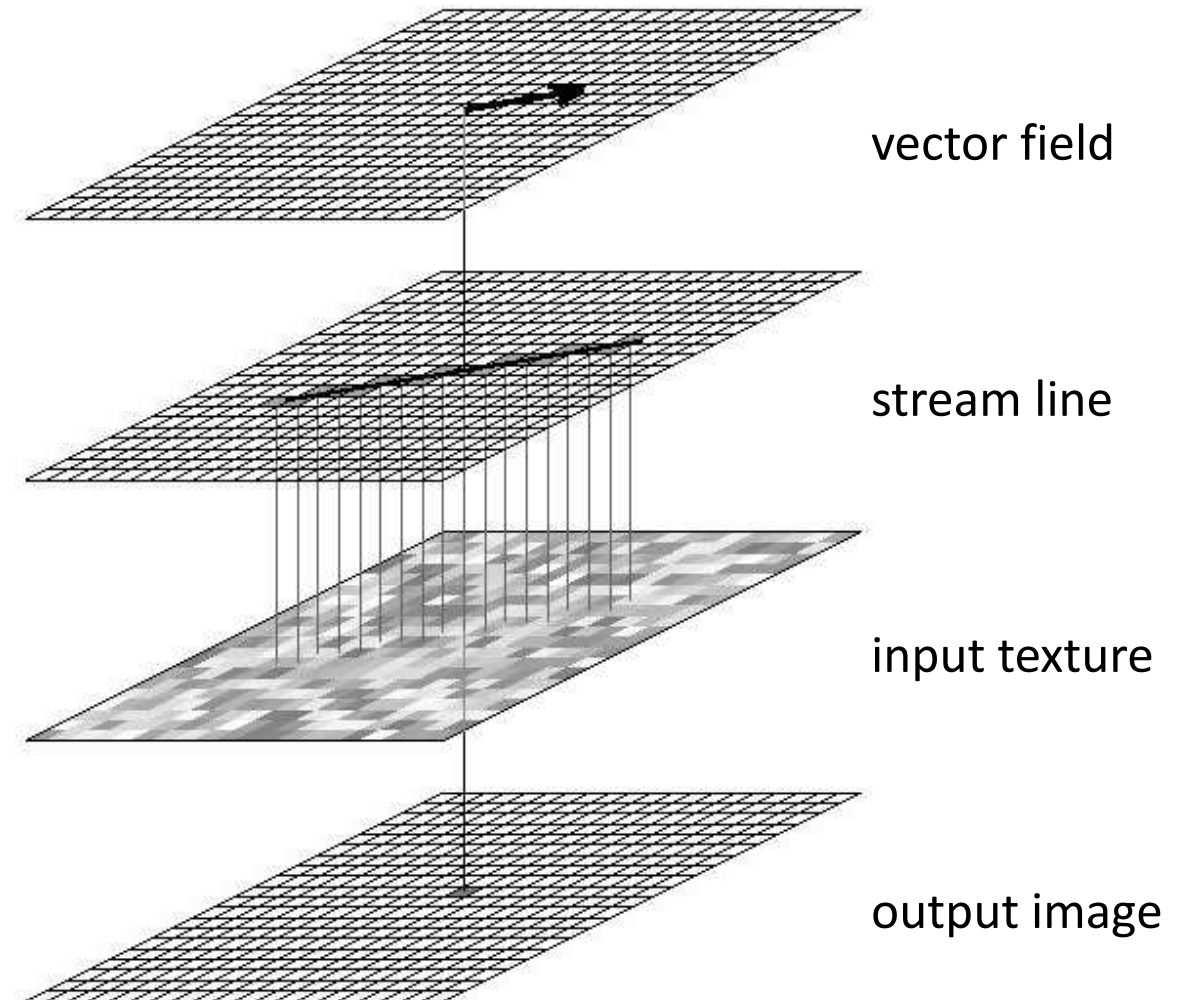
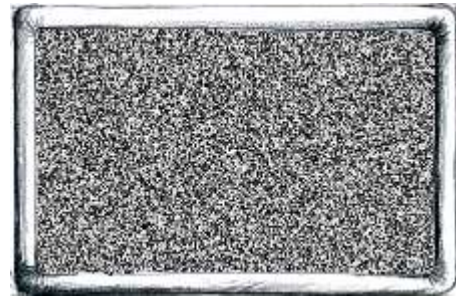


Illustration: Algorithm for Line Integral Convolution



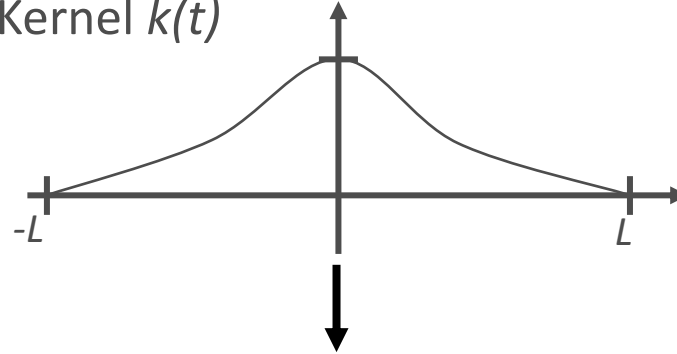
Input texture
(noise)



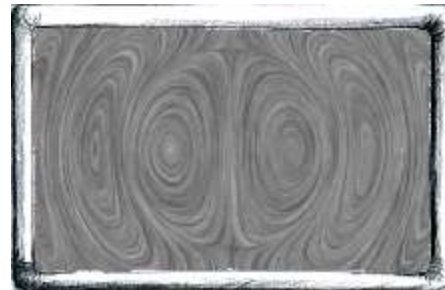
Vector field

Convolution

Kernel $k(t)$



$$\int_{-L}^L k(t) dt = 1$$

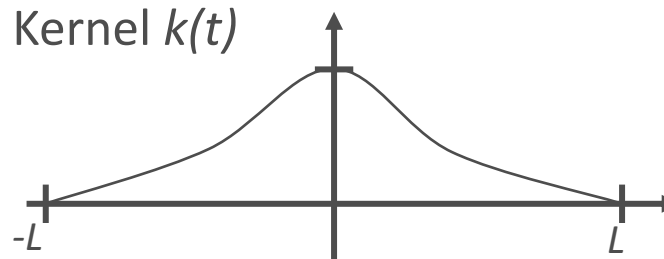


Final image

Details: Algorithm for Line Integral Convolution

- Steps for computing **Line Integral Convolution** at (x_0, y_0)
 - Integrate a stream line $\varphi_0(t)$ starting at the point $\varphi_0(0)=(x_0, y_0)$ for length L in forward and backward direction, respectively
 - $T(x, y)$ is the randomly generated input texture
 - Compute the pixel intensity as:
$$I(x_0, y_0) = \int_{-L}^L k(-t) \cdot T(\varphi_0(t)) dt$$

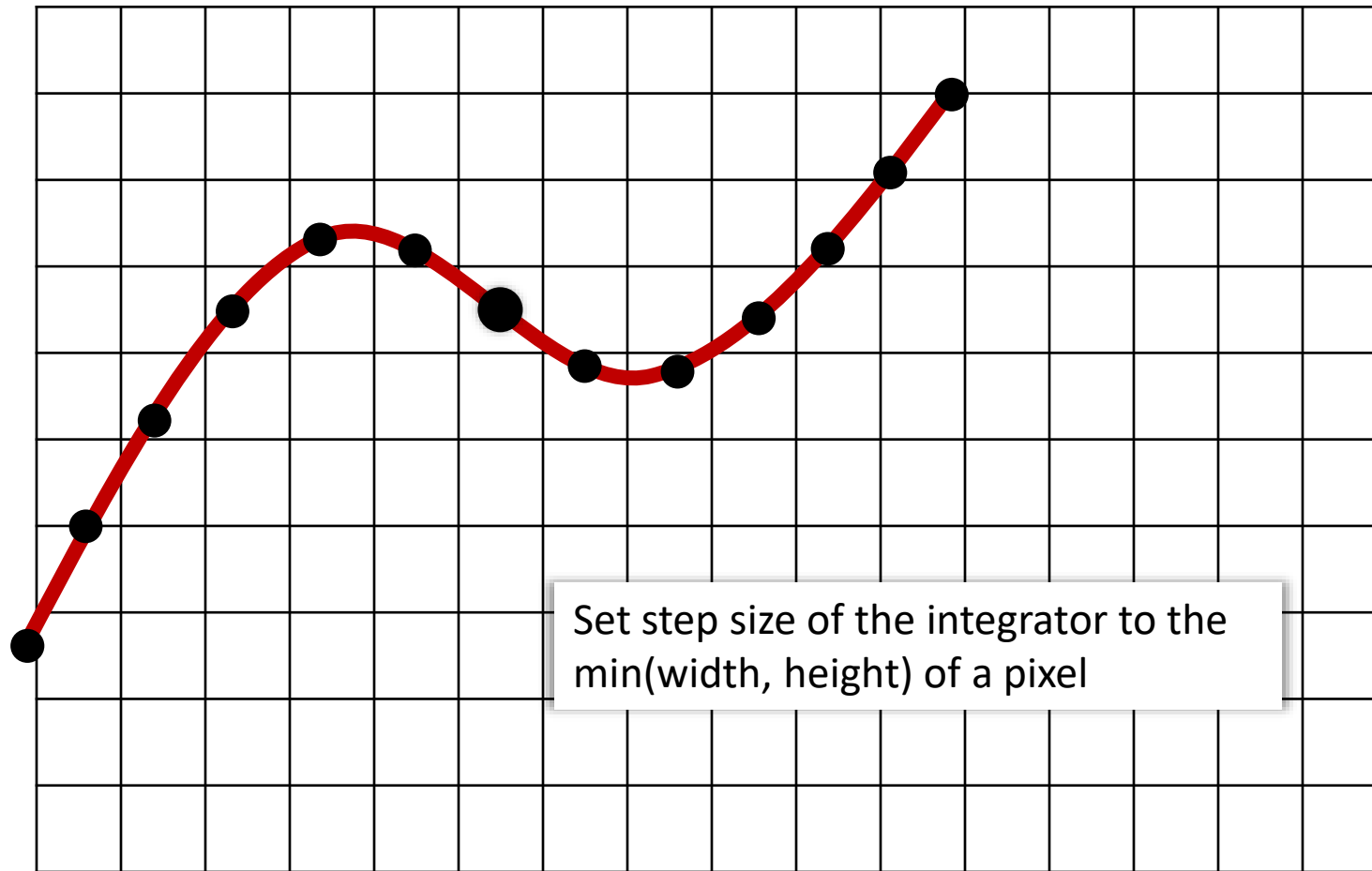
*convolution of the input texture T along the stream line $\varphi_0(t)$ with a kernel $k(t)$, which has length $2*L$.*
 - Kernel:
 - Finite support $[-L, L]$
 - Normalized $\int_{-L}^L k(t) dt = 1$
 - Symmetric $k(-t) = k(t)$



Discretizing the Integral

Discretization of line convolution integral: $I(x_0, y_0) = \int_{-L}^L k(-t) \cdot T(\varphi_0(t)) dt$

- sample the stream line equidistantly
- evaluate the random texture as a bilinear scalar field



Choice of Kernel

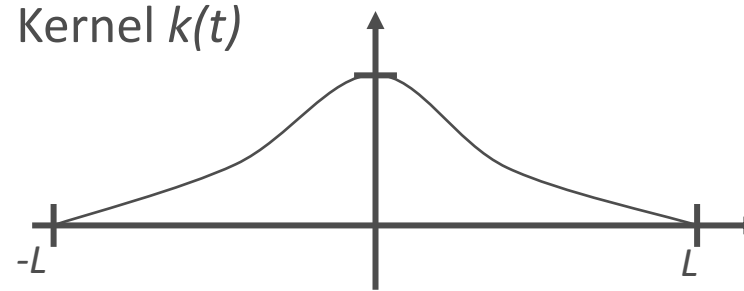
- Popular **types of kernels**:

- Gaussian kernel
- Triangle kernel
- Box kernel

- Amounts to taking the arithmetic mean

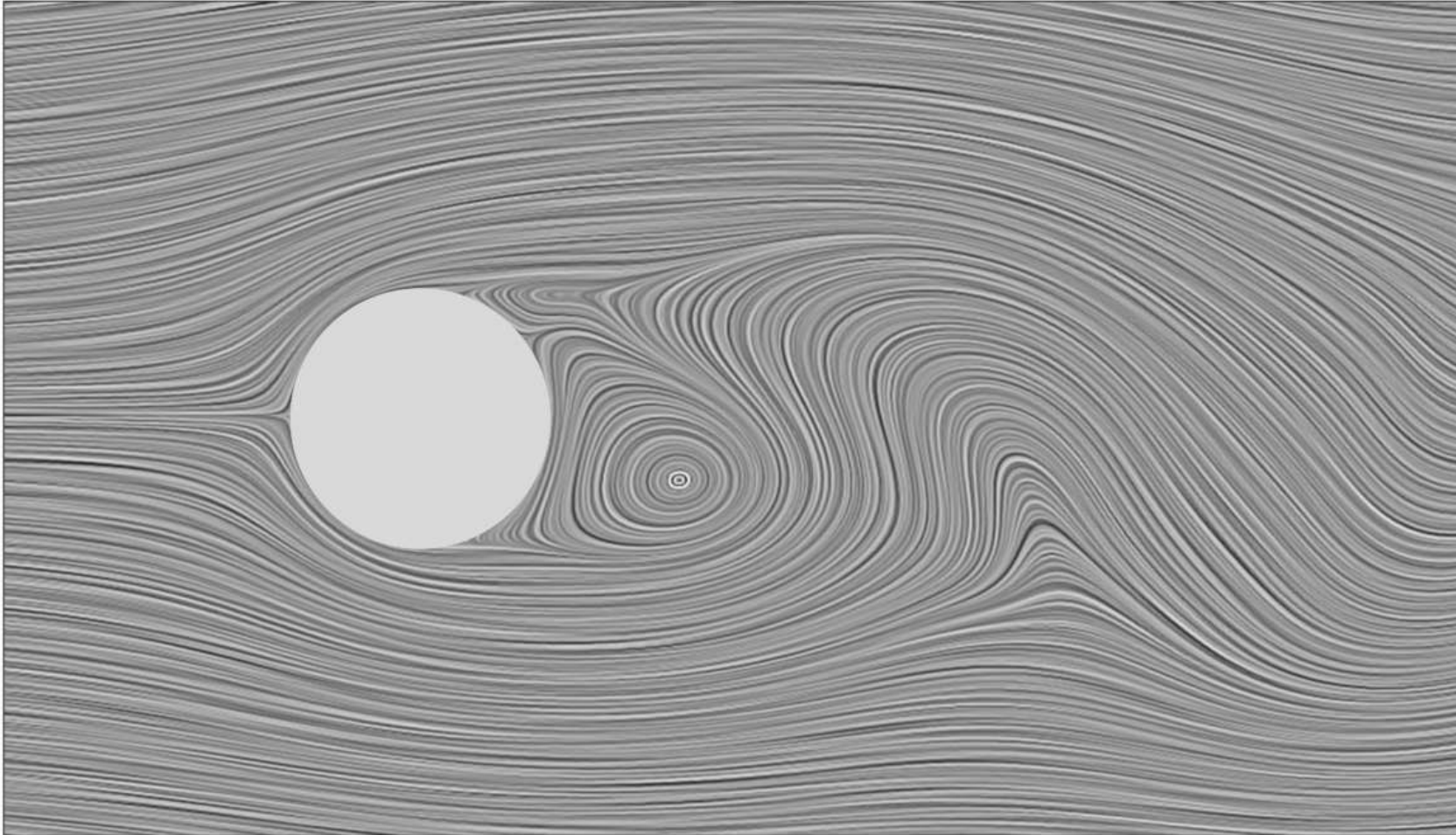
- Effect of **length of kernel**:

- Longer kernel: Longer lines, less contrast
- Smaller kernel: Shorter lines, more contrast



$$\int_{-L}^L k(t) dt = 1$$

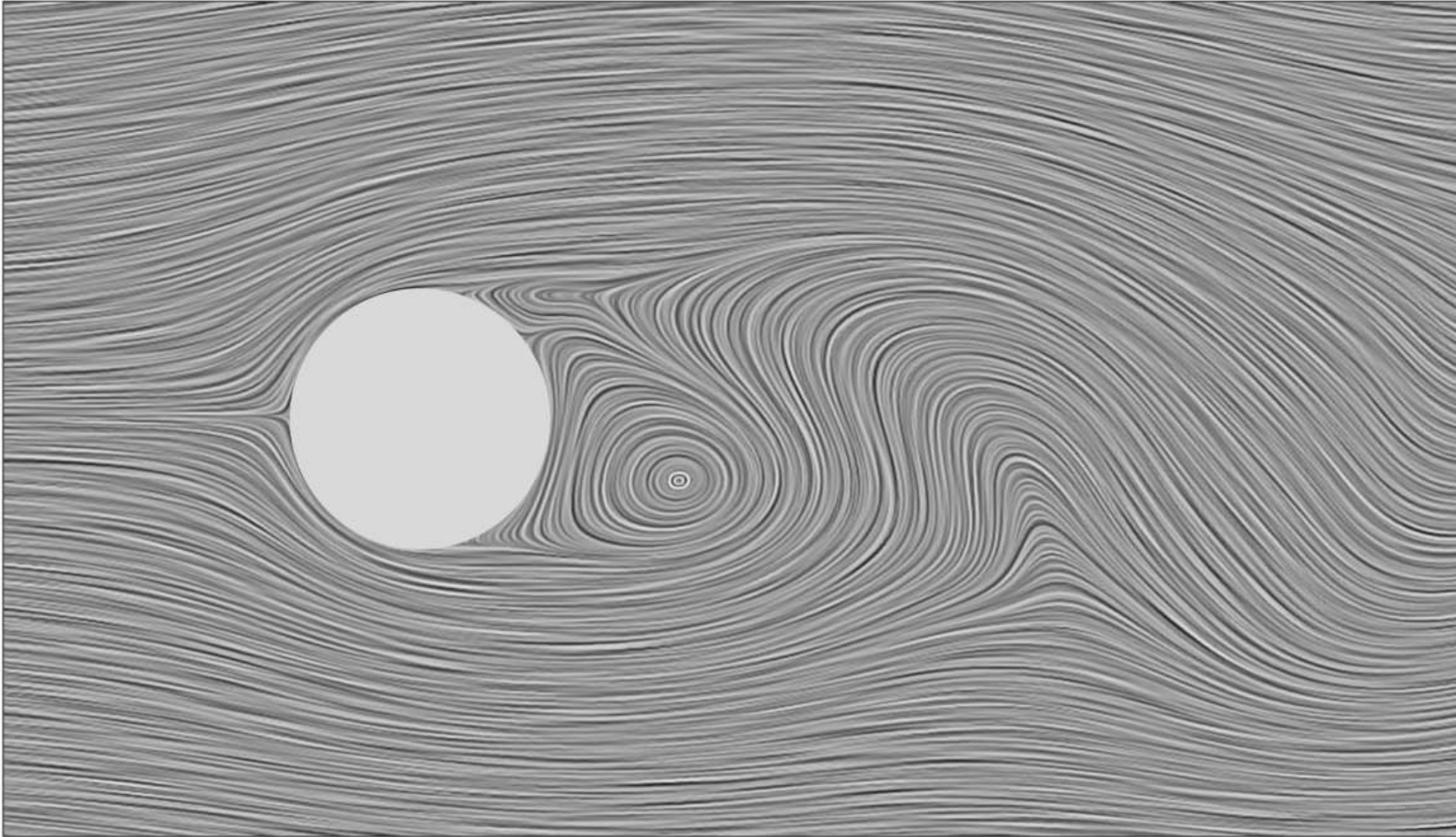
Effect of Filter Length in LIC



Filter length influences the quality of LIC images
filter length = 100

2D flow behind a cylinder

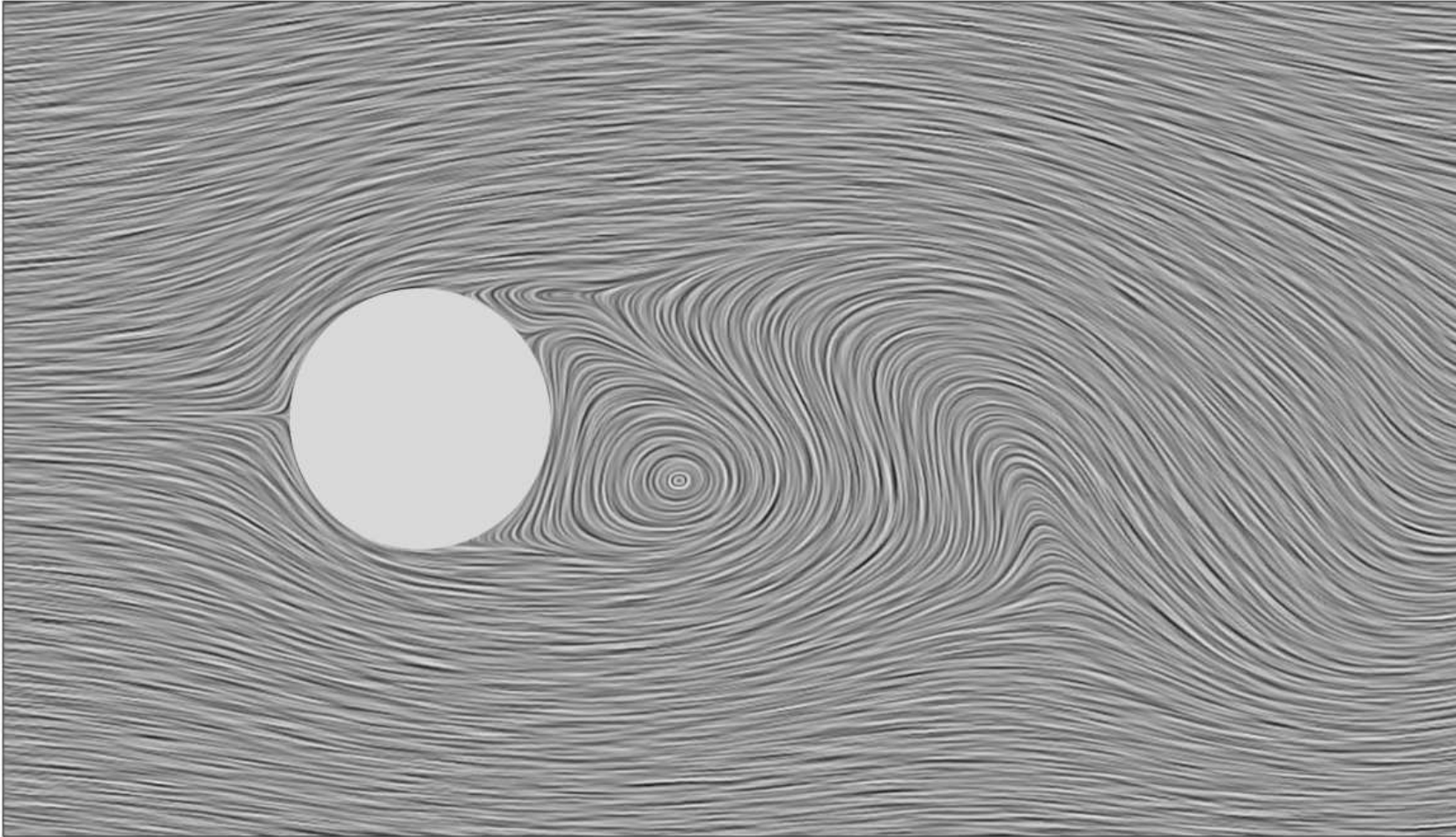
Effect of Filter Length in LIC



Filter length influences the quality of LIC images
filter length = 50

2D flow behind a cylinder

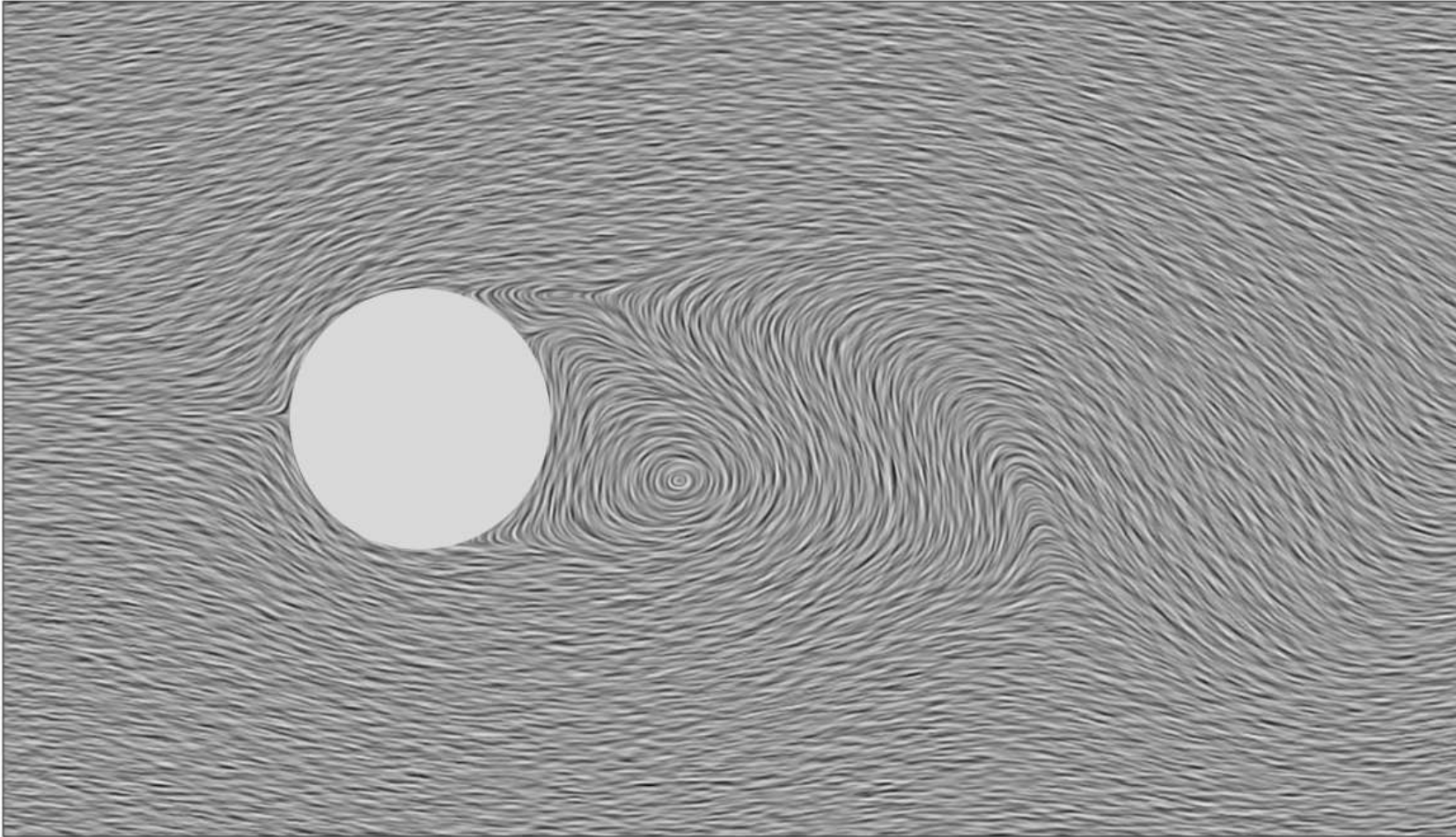
Effect of Filter Length in LIC



Filter length influences the quality of LIC images
filter length = 25

2D flow behind a cylinder

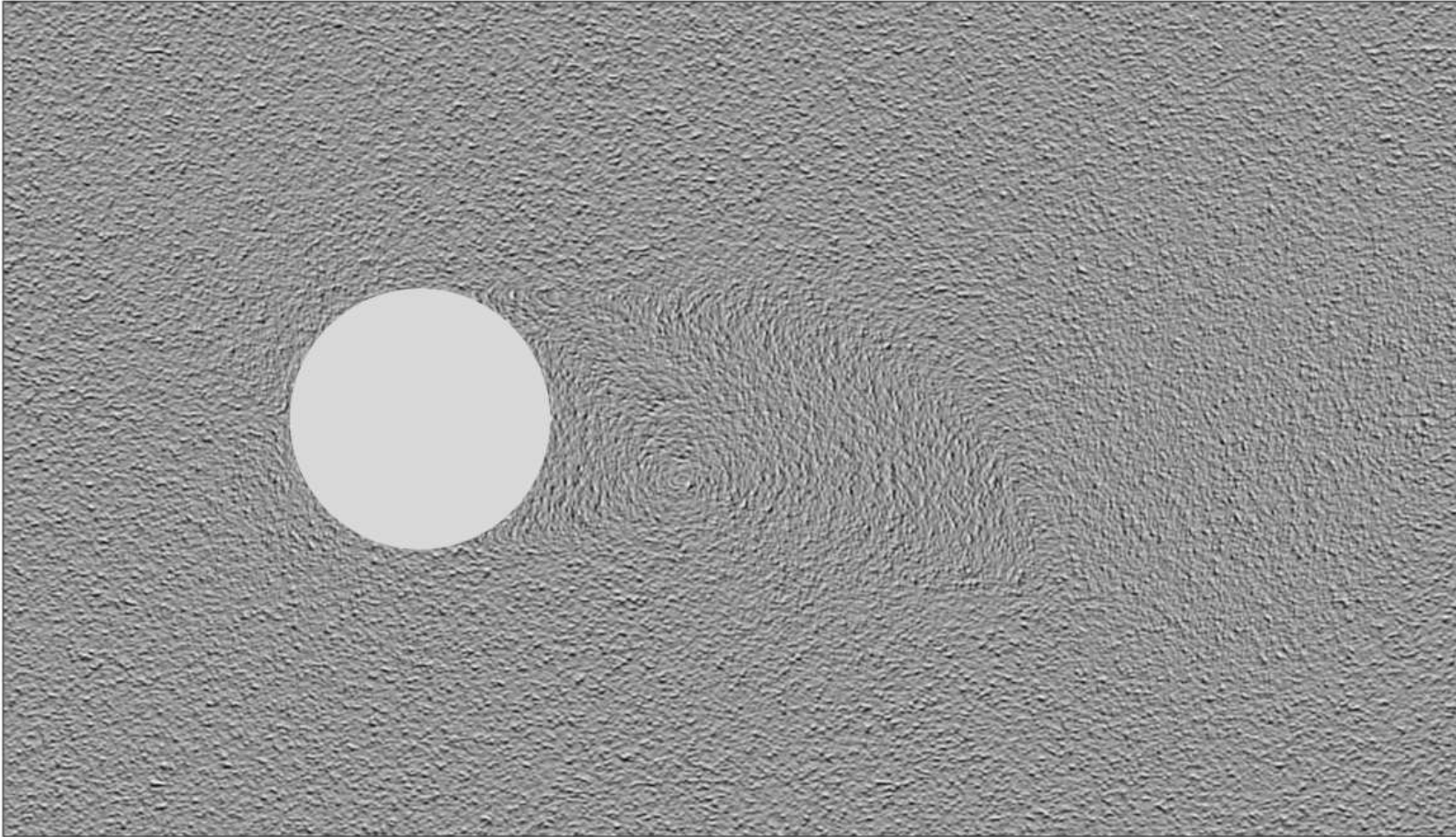
Effect of Filter Length in LIC



Filter length influences the quality of LIC images
filter length = 10

2D flow behind a cylinder

Effect of Filter Length in LIC



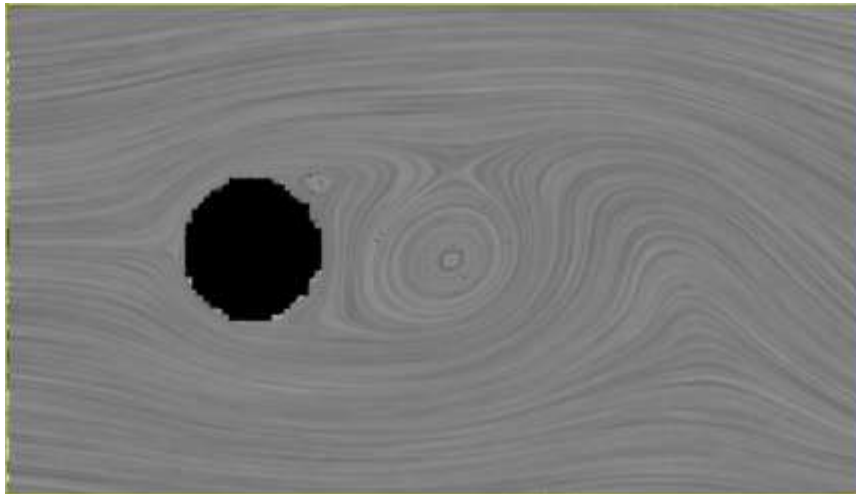
Filter length influences the quality of LIC images
filter length = 1

2D flow behind a cylinder

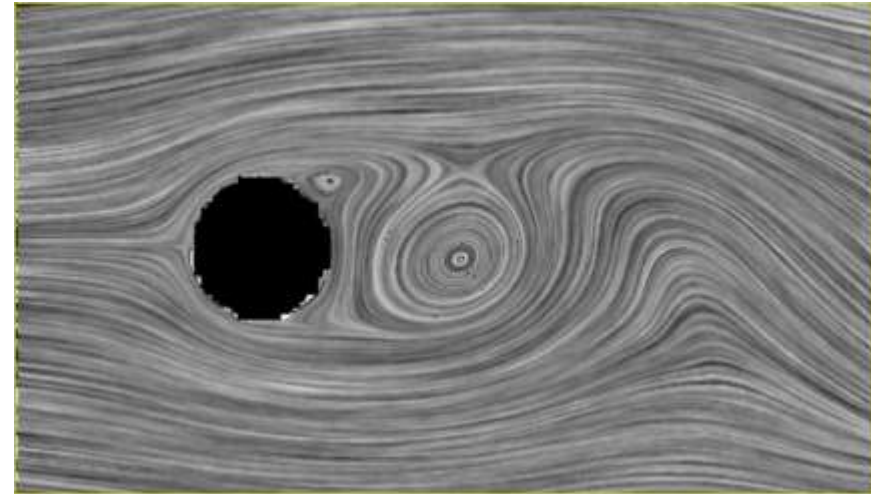
Contrast Enhancement

- *Problem:* Convolution **reduces image contrast**
 - Enhance contrast in final image
 - *Example:* linearly map mean/standard deviation μ/σ of pixel intensities I_p after convolution to the desired values μ'/σ' :

$$I'_p = \mu' + \frac{\sigma'(I_p - \mu)}{\sigma}$$



Result after convolution



Result after contrast enhancement

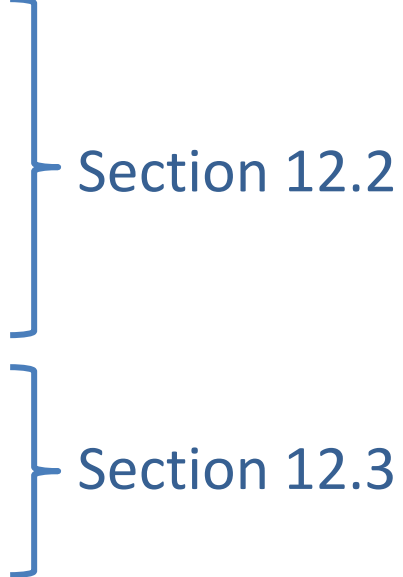
Summary: Line Integral Convolution

- LIC produces dense image-based flow visualizations by **convolving a noise texture** with a one-dimensional kernel **along the stream lines**
 - Support (length) of the kernel determines our ability to visually follow stream lines over larger distances
 - Image contrast is reduced by the convolution and should be restored again for the final rendering

Section 12.2: Extensions of LIC

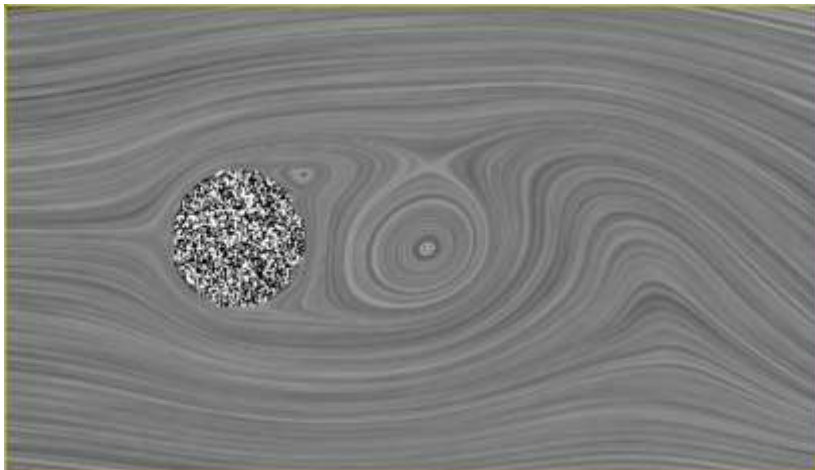
LIC: Extensions and Improvements

Basic LIC has been extended and improved in the following ways:

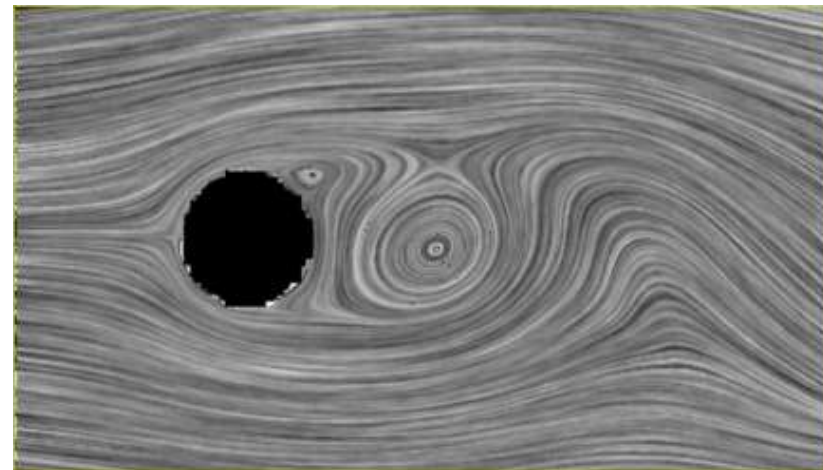
- faster computation (FastLIC)
 - color coding / color weaving
 - adding flow orientation (OLIC)
 - LIC on surfaces
 - LIC for 3D flows
- 
- Section 12.2
- Section 12.3

Fast LIC

- **Standard LIC**
 - Stream line integration started at each pixel
 - Convolution (integral) is computed at each pixel
- **Idea of Fast LIC** [Stalling/Hege 1995]:
 - Make stream lines as long as possible
 - Reuse these stream lines for all pixels that they cover
 - Start new streamlines only at pixels that have not yet been covered



Classic: 131072 stream lines, 9 seconds



FastLIC: 6186 stream lines, <1 second

Illustration: Basic Idea of FastLIC

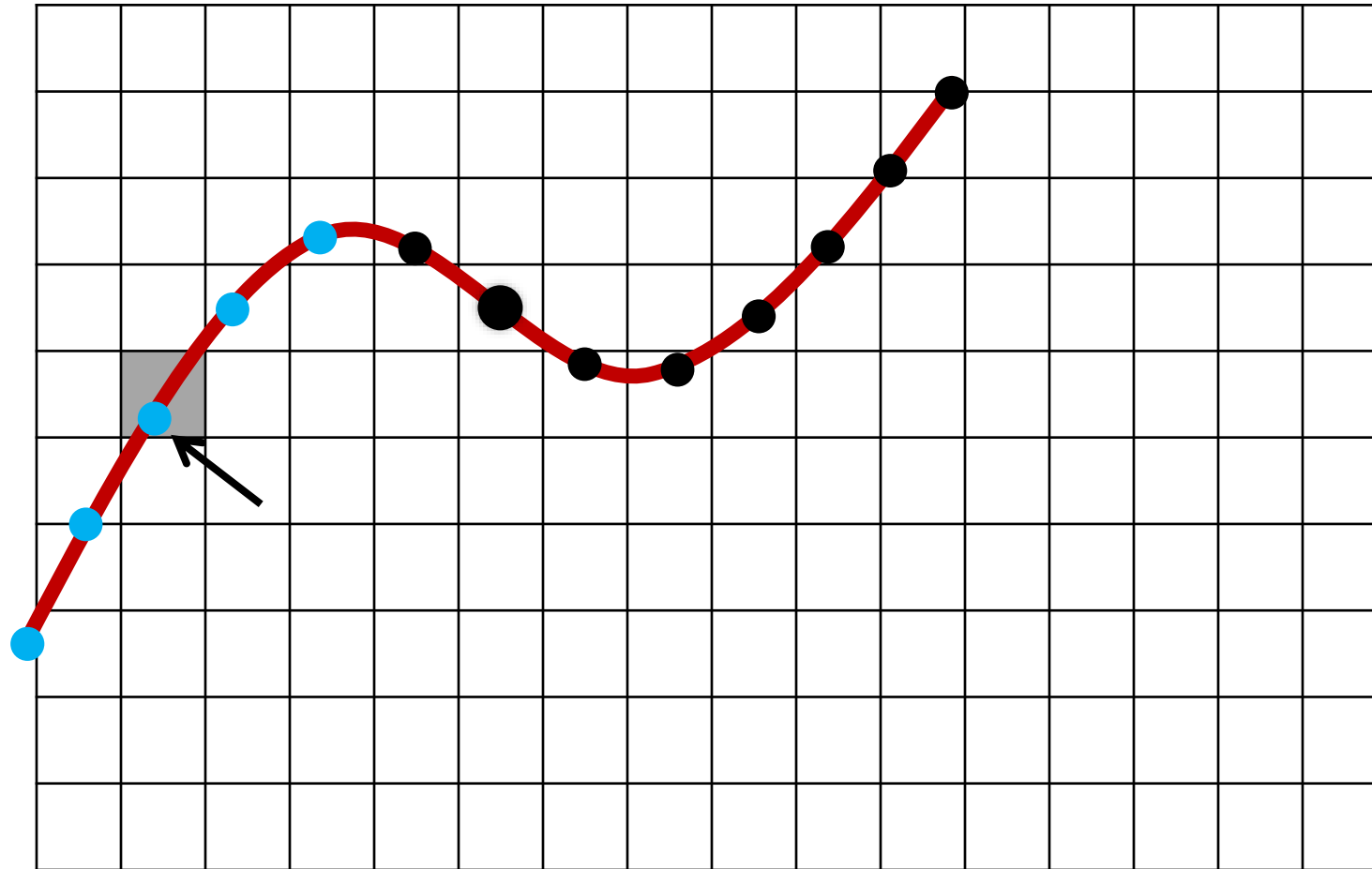
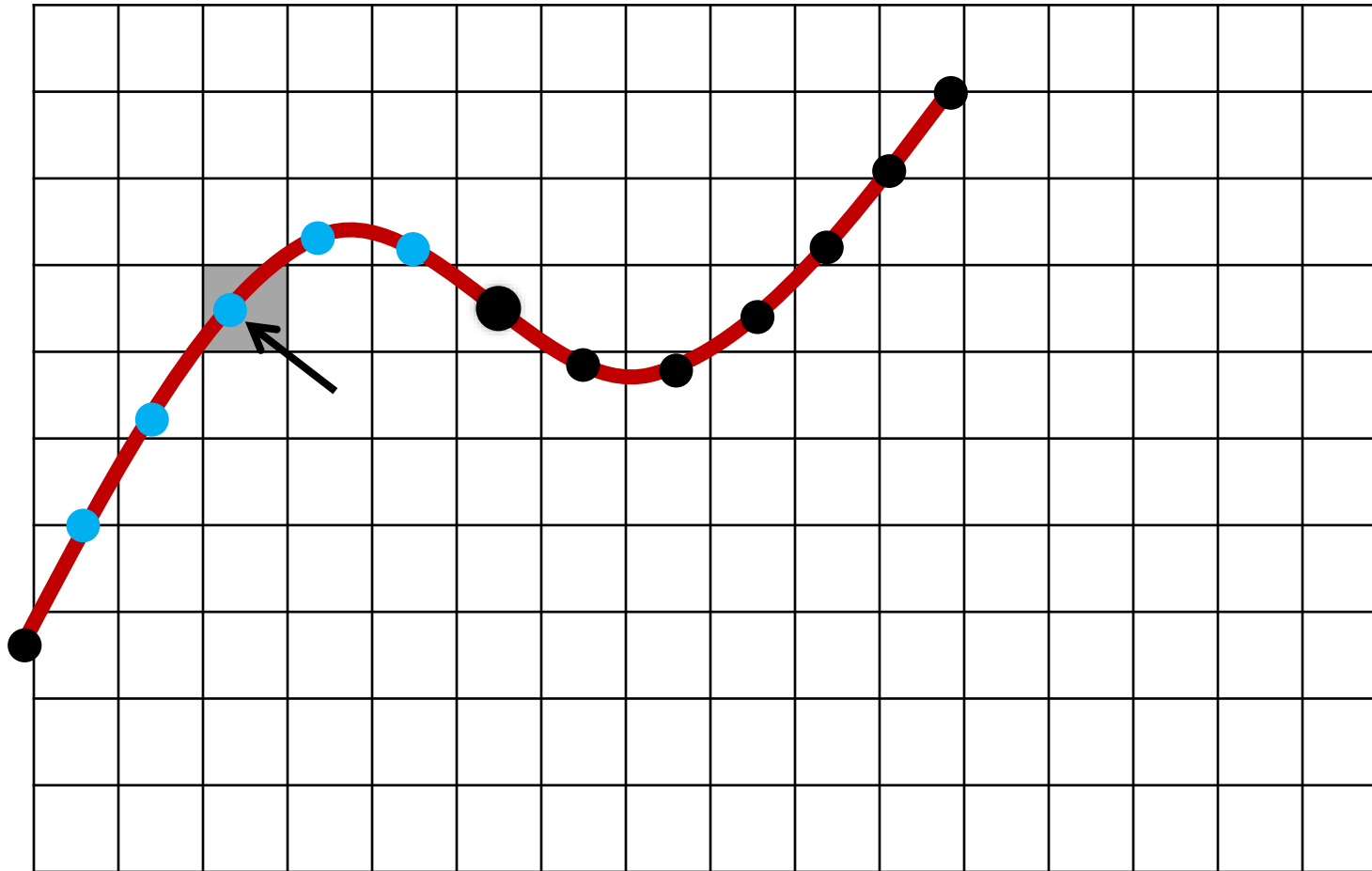


Illustration: Basic Idea of FastLIC

- *Additional benefit:* For many kernels, we do not even need to re-compute the whole Riemann sum



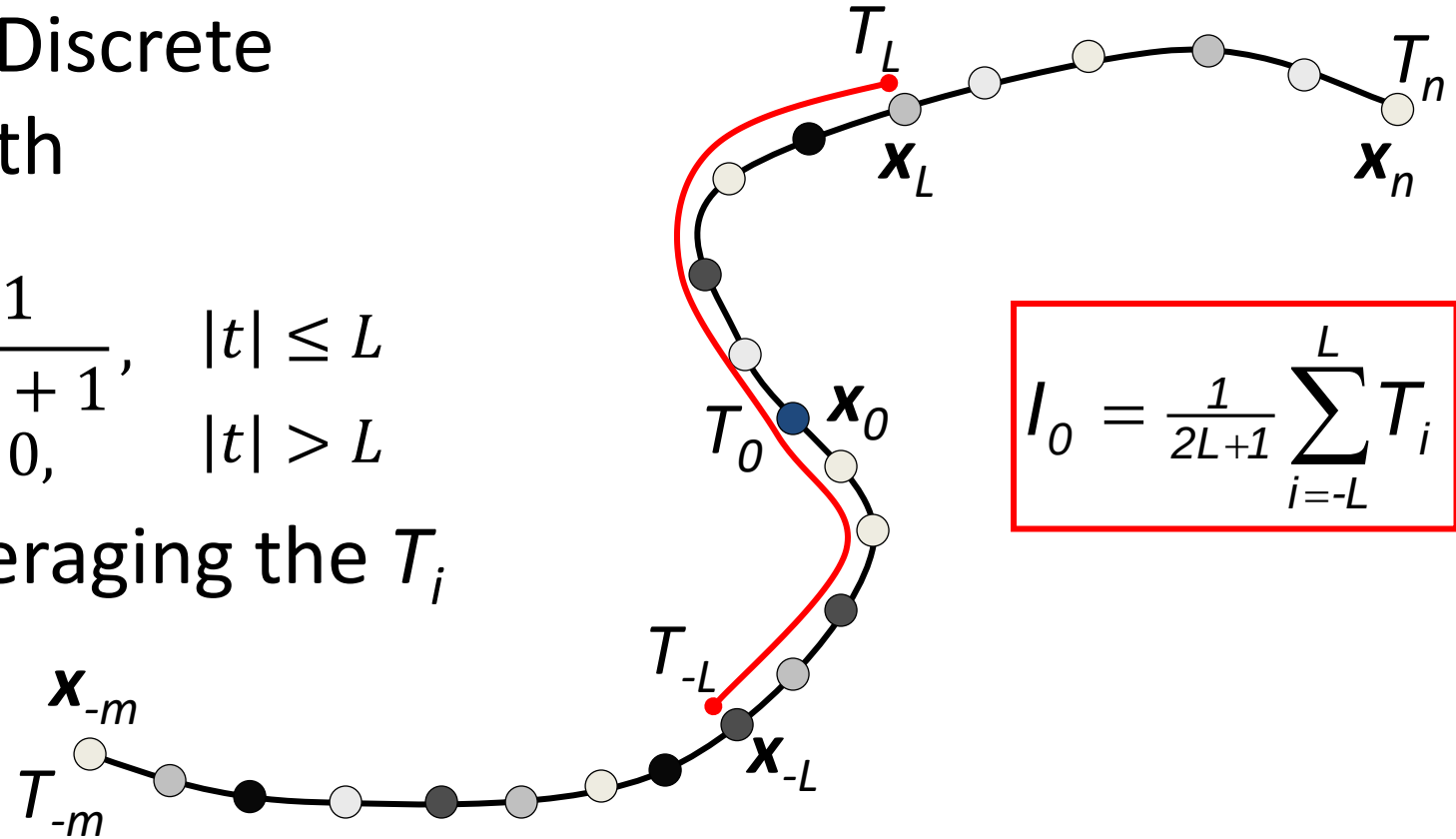
Incremental Integration in Fast LIC

- Fast LIC: incremental integration

- Simplest case: Discrete convolution with box filter

$$k(t) = \begin{cases} \frac{1}{2L+1}, & |t| \leq L \\ 0, & |t| > L \end{cases}$$

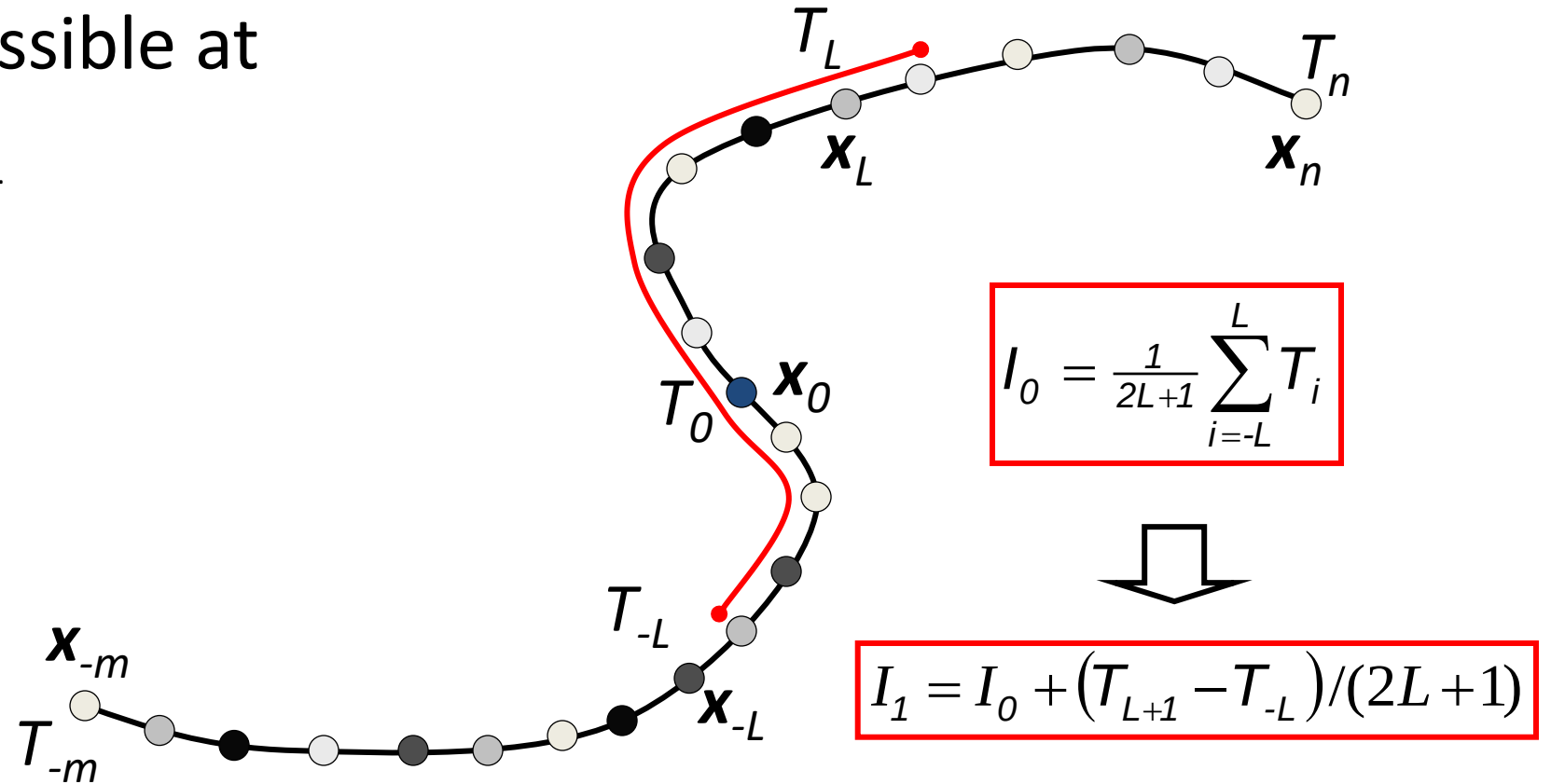
amounts to averaging the T_i



$$I_0 = \frac{1}{2L+1} \sum_{i=-L}^L T_i$$

Incremental Integration in Fast LIC

- Fast LIC: incremental integration
 - Fast update possible at next position I_1



Fast LIC: Pseudocode

- Data structure for output: Texture + count arrays of equal dimensions
 - Texture = gray-scale output
 - Count = number of streamlines passing through that pixel

For each pixel **p** in output image

If (Count(**p**) < 1) then

Initialize streamline computation with

$\mathbf{x}_0$ = center of **p**

Compute convolution $I(\mathbf{x}_0)$

Add result to pixel **p** and increment Count(**p**)

For $m = 1$ to Limit M

Incremental convolution for $I(\mathbf{x}_m)$ and $I(\mathbf{x}_{-m})$

Add results to pixels containing $\mathbf{x}_m$ and $\mathbf{x}_{-m}$
and increment Count value of those pixels

End for

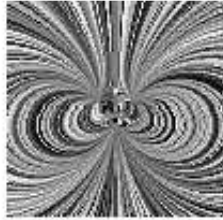
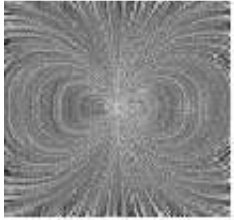
End if

End for

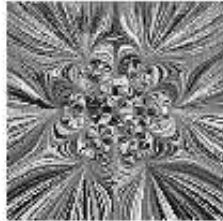
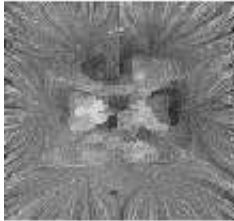
Normalize all pixels according to Count

Integrate & Draw

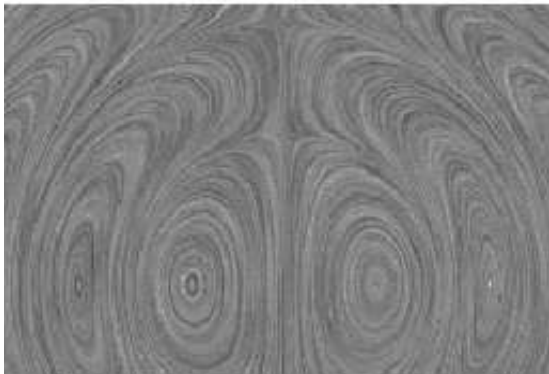
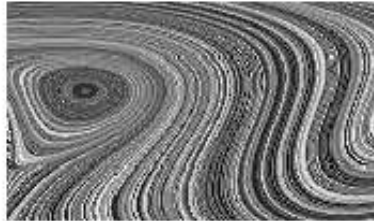
a)



b)



c)



Idea by [Pérez, 1998]:

Instead of convolution of an input texture, assign a random gray value to each stream line

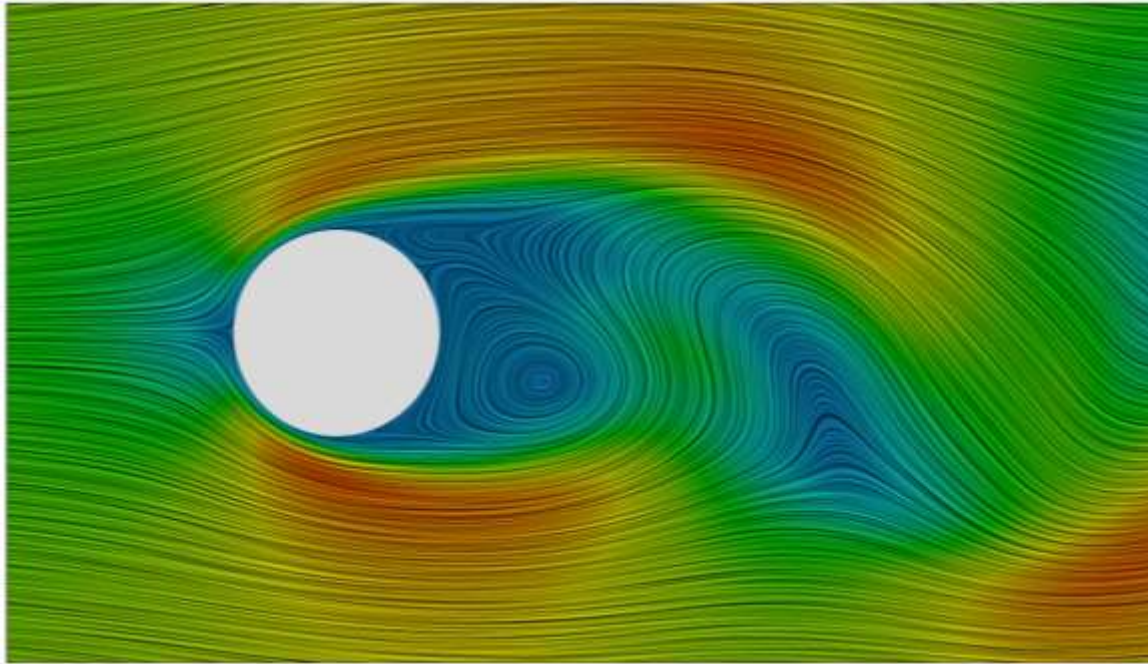
- Average gray values if multiple stream lines enter the same pixel

Left: LIC

Right: Integrate & Draw

LIC with Color Coding

- Standard LIC does not use the color channel
 - Can use color to encode scalar quantities



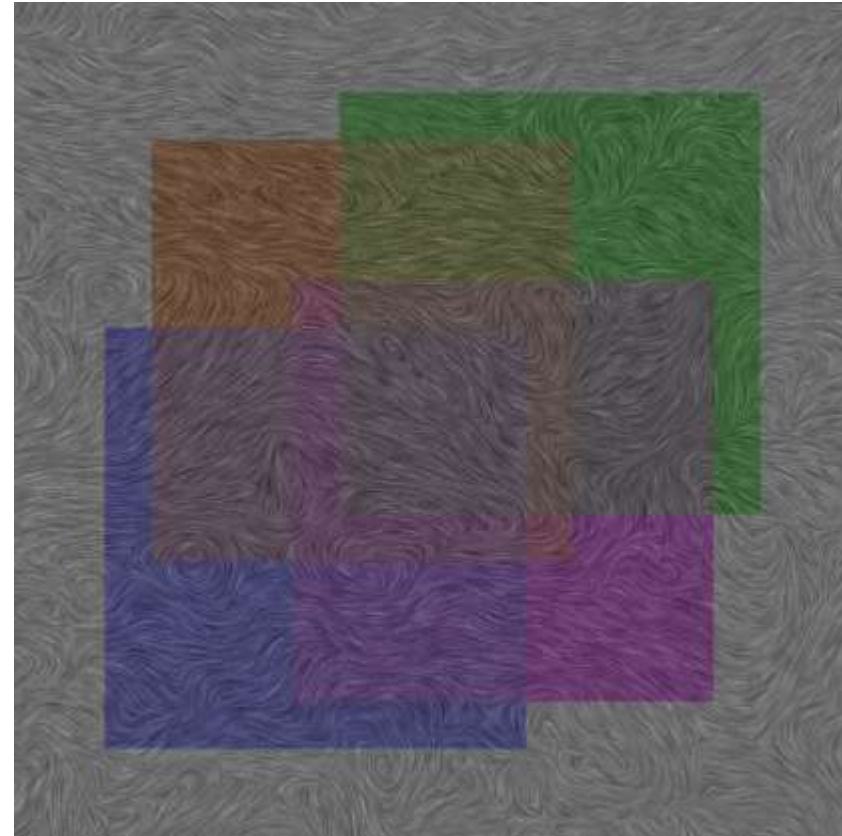
2D flow behind a cylinder

Velocity magnitude
encoded using color

Color Weaving

- **Color Weaving** [Urness et al., Vis 2003]:
 - Method to encode different scalar quantities in a single LIC image

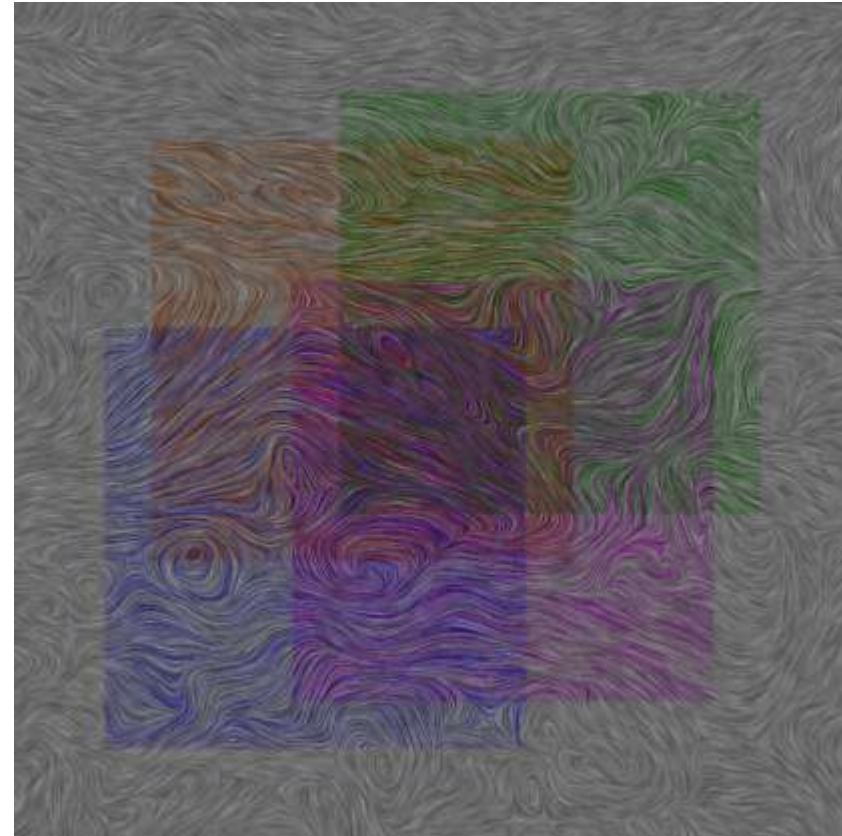
Four artificially defined, mutually overlapping regions, overlaid on a LIC image. The color combinations in the regions of overlap are obtained by averaging in RGB colorspace.



Color Weaving

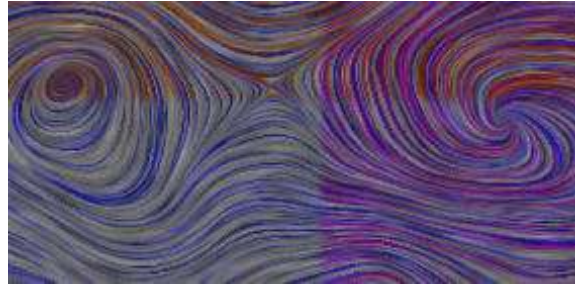
- **Color Weaving** [Urness et al., Vis 2003]:
 - Method to encode different scalar quantities in a single LIC image

The same synthetic data visualized with color weaving. Colors are continuous along individual streamlines within each region, and it remains possible to perceive individual component colors in areas of high overlap.

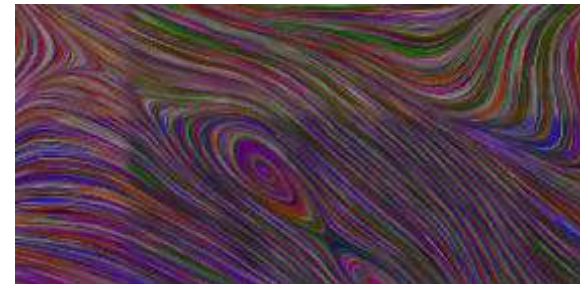
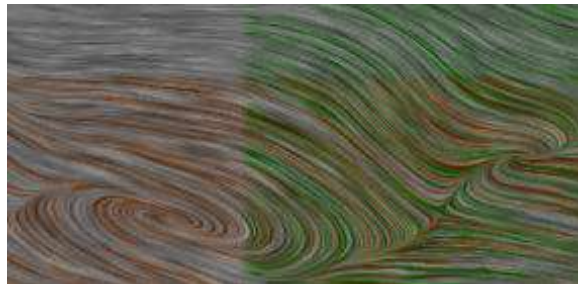


Basic Idea of Color Weaving

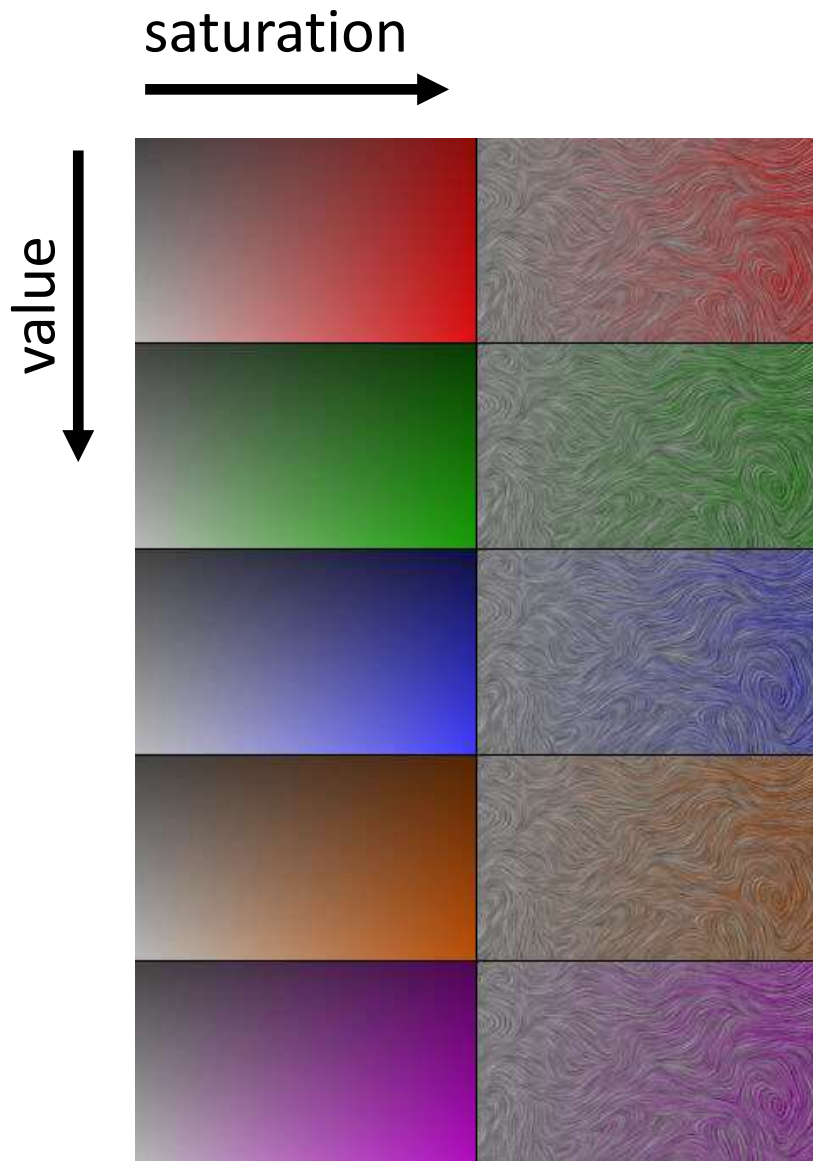
- Color Weaving visualizes multiple scalar fields on top of LIC by encoding them into the color of neighboring stream lines



Close-ups



Color Maps for Color Weaving

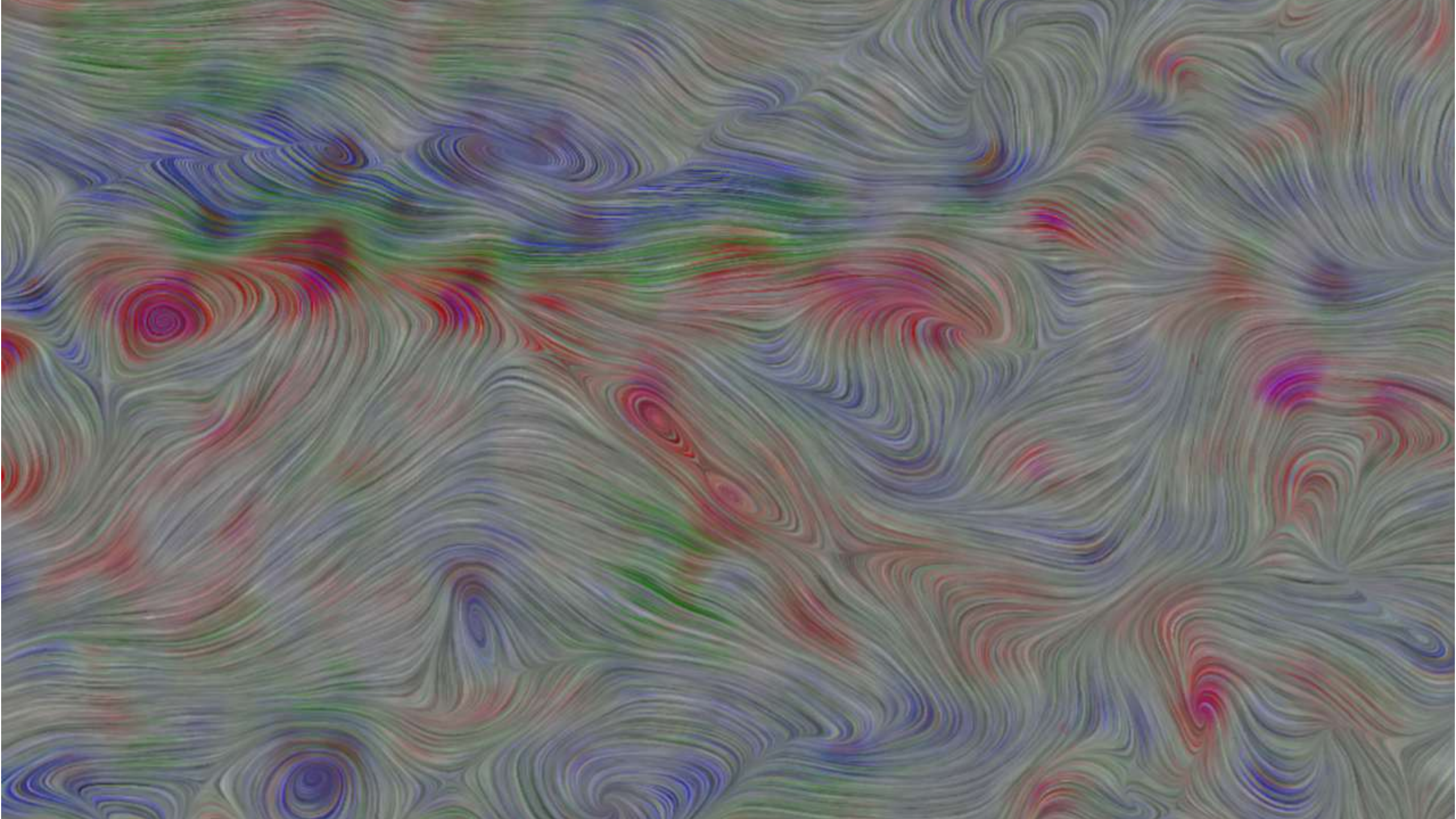


Based on HSV color space:

- 5 saturated and isoluminant base colors
- Select colors that are easy to discriminate
- Create two-dimensional color maps:
 - Vary saturation along horizontal axis
 - Vary value along vertical axis

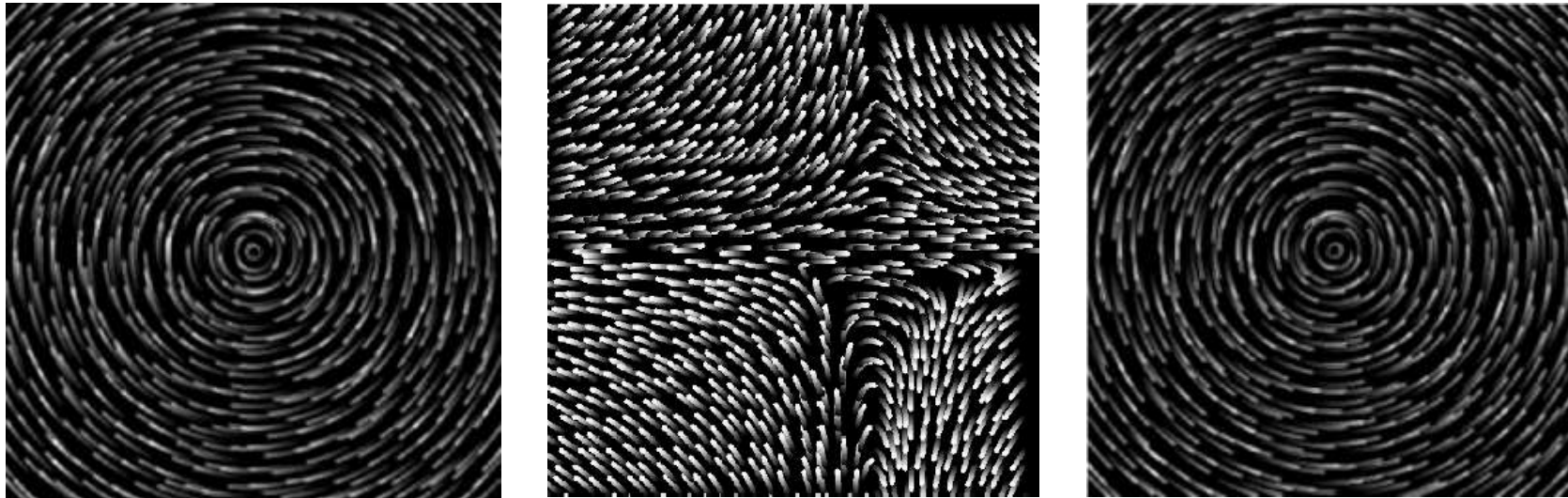
Implementation of Color Weaving

- Each scalar quantity gets its own colormap
- **Final pixel color** (HSV color space):
 - **Hue:** scalar quantity to be encoded, i.e., its colormap (unique color per streamline)
 - **Saturation:** value of the scalar quantity
 - **Value:** gray value obtained from LIC
- Implementation based on **fast LIC**
 - increments a color index when a new stream line is started
 - Hue and saturation are determined by the first streamline that enters a pixel



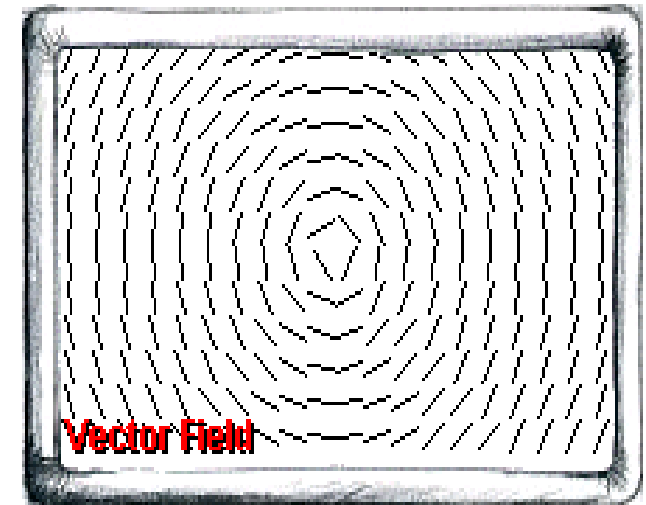
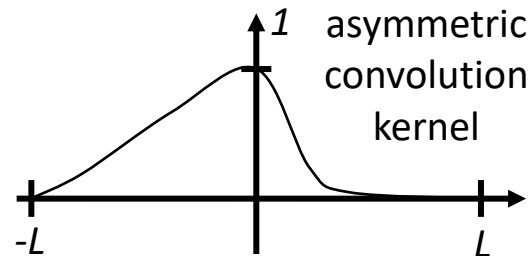
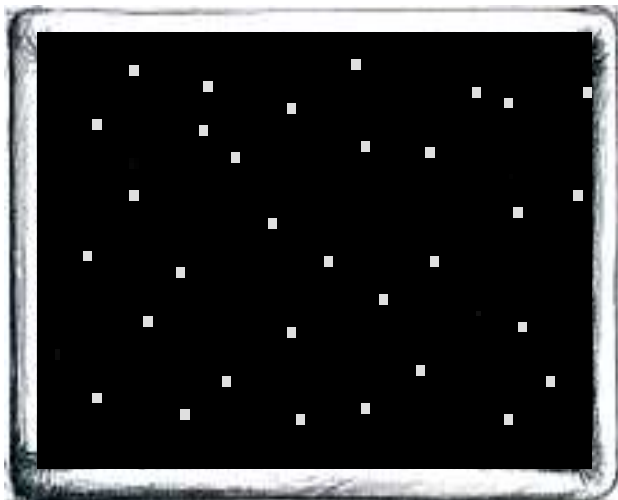
Oriented Line Integral Convolution

- Oriented LIC (OLIC) [Wegenkittl et al. 1997] additionally conveys the orientation of the vector field

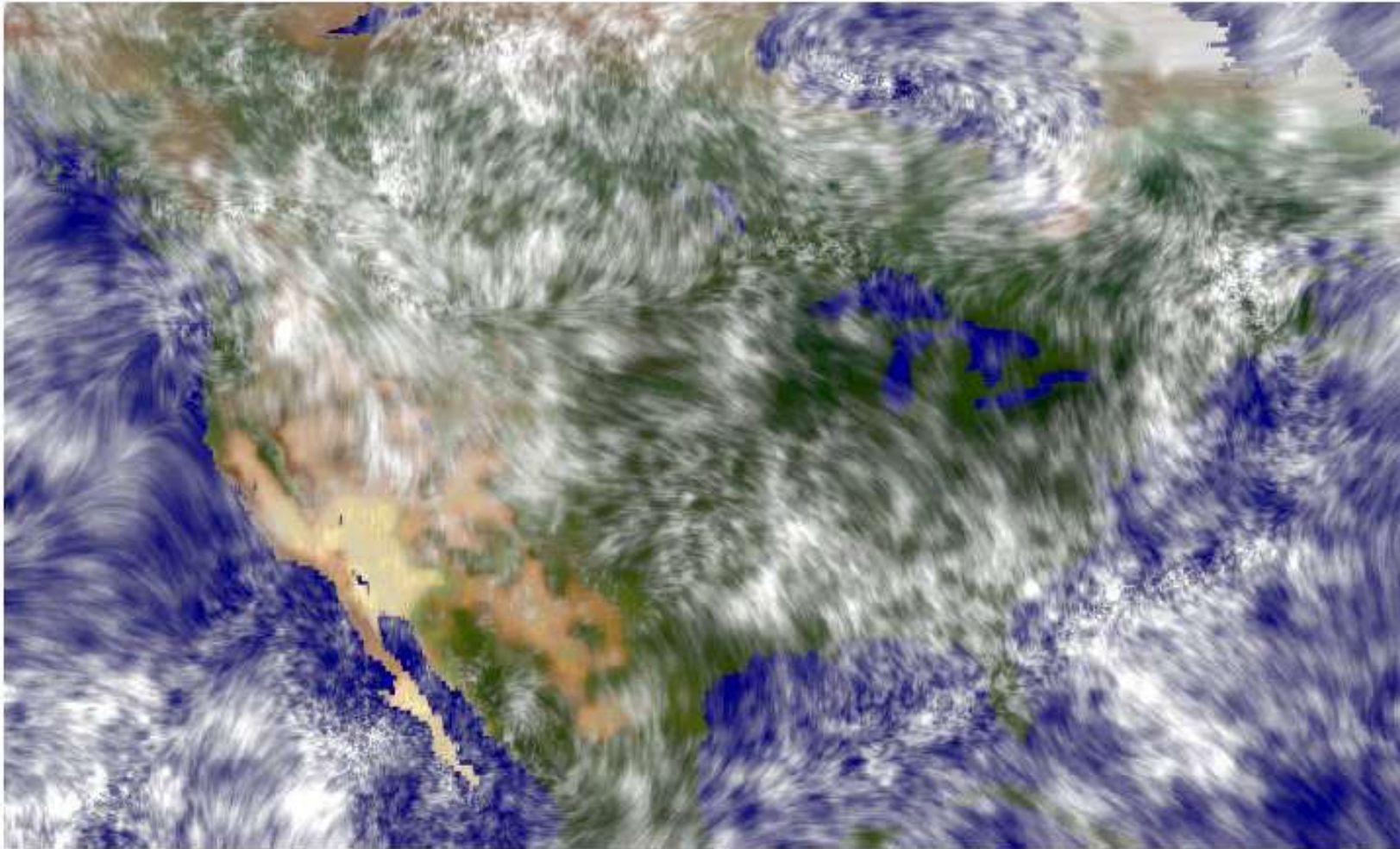


Oriented Line Integral Convolution

- Oriented LIC (OLIC) [Wegenkittl et al. 1997]:
 - Visualizes **orientation** (upstream vs. downstream) by using an **asymmetric** convolution kernel
 - Uses a **sparse texture** so that we can observe how individual “drops of ink” spread out



LIC: Adaptive Kernel Length



Length of convolution integral depends on
magnitude of vector field

Fun With LIC



Added motion blur by means of variable length LIC

Summary: Extensions of Line Integral Convolution

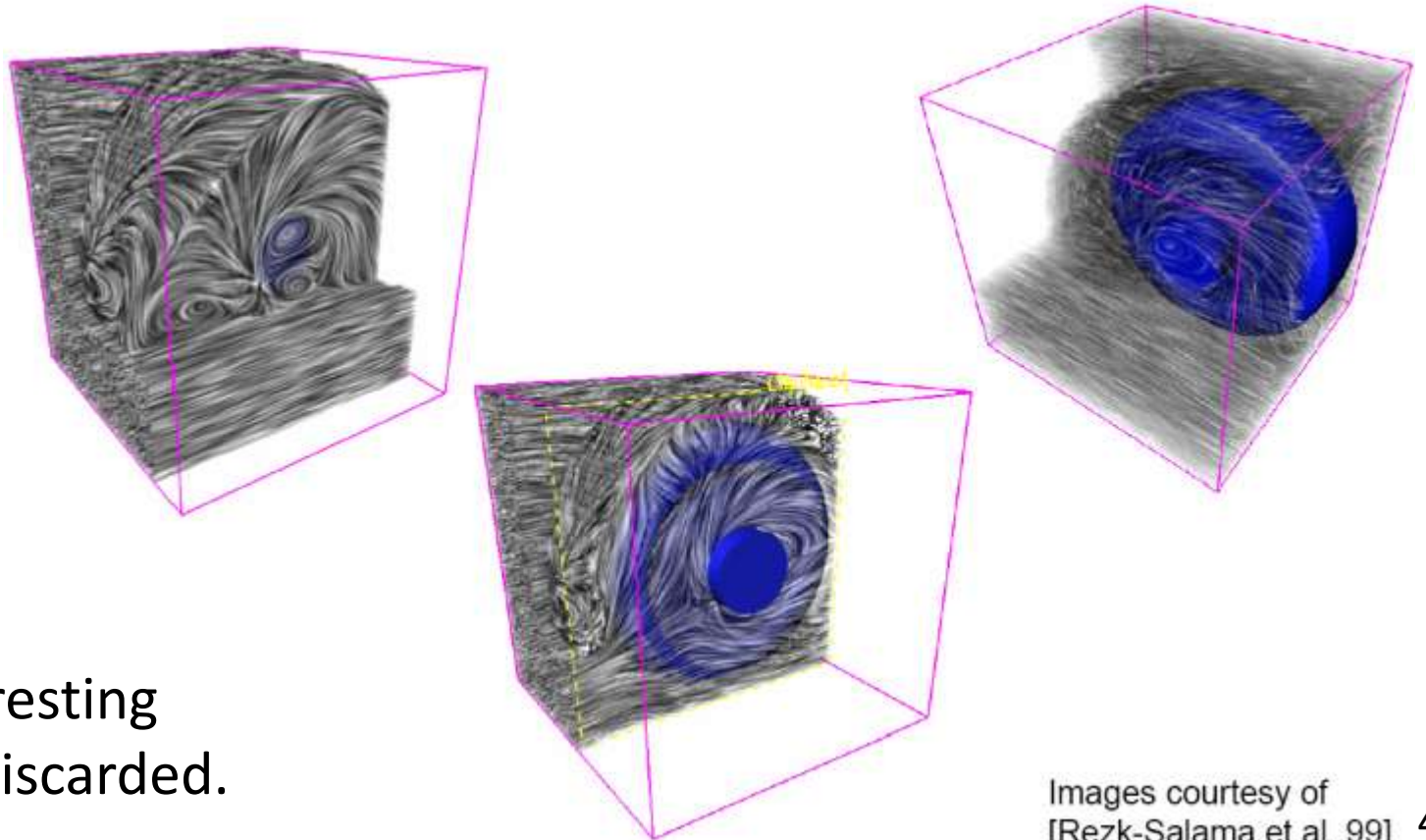
- Integrating long streamlines and re-using them for incremental computation permits **fast LIC**
- Additional quantities can be **color coded**
 - Based on fast LIC, **color weaving** visualizes multiple scalar fields in the same image, but on different streamlines to avoid color blending
- Combining asymmetric kernels with sparse noise textures results in **oriented LIC**

Section 12.3:

3D LIC / LIC on Surfaces

LIC for 3D Flows

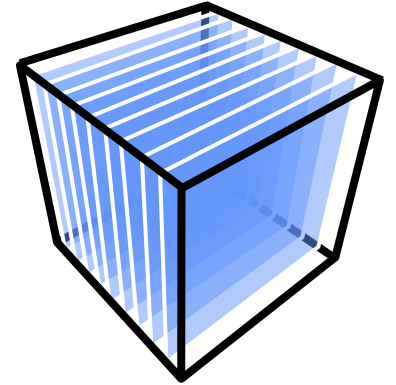
- Generalization to 3D straightforward
 - Use 3D texture and 3D stream line integration
 - Fast LIC highly recommended
- Problem: Rendering!



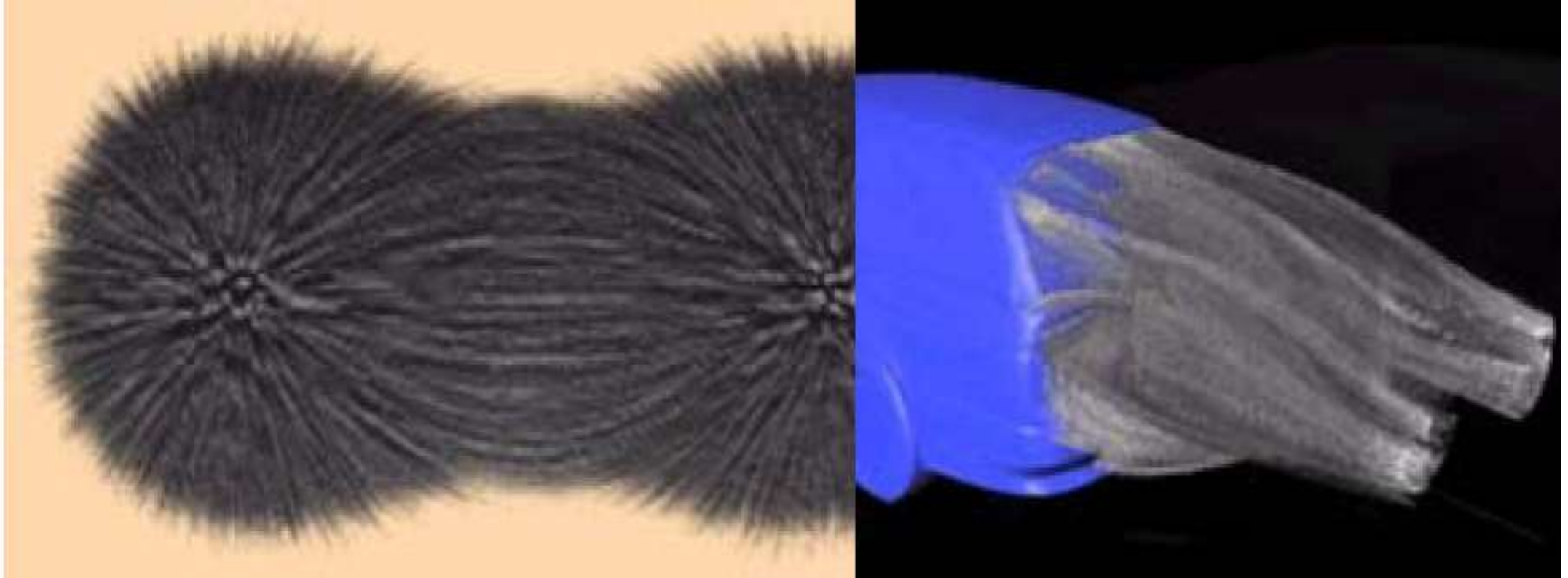
3D LIC can only reveal interesting structures if some data is discarded.

Rendering 3D LIC

- Volume Render LIC volume texture as explained in Chapter 10, e.g., via
 - Ray casting
 - Slice Based Volume Rendering
- Needed:
 - Transfer Function that makes part of the volume (semi-)transparent, e.g., based on vortex detectors

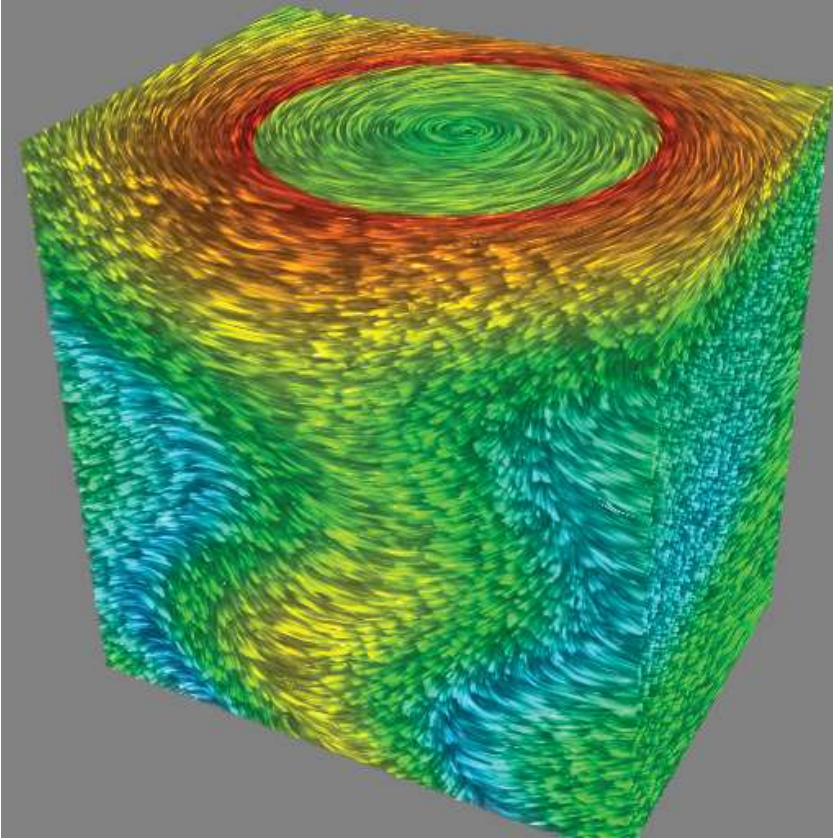


3D LIC: Opacity from Derived Quantities

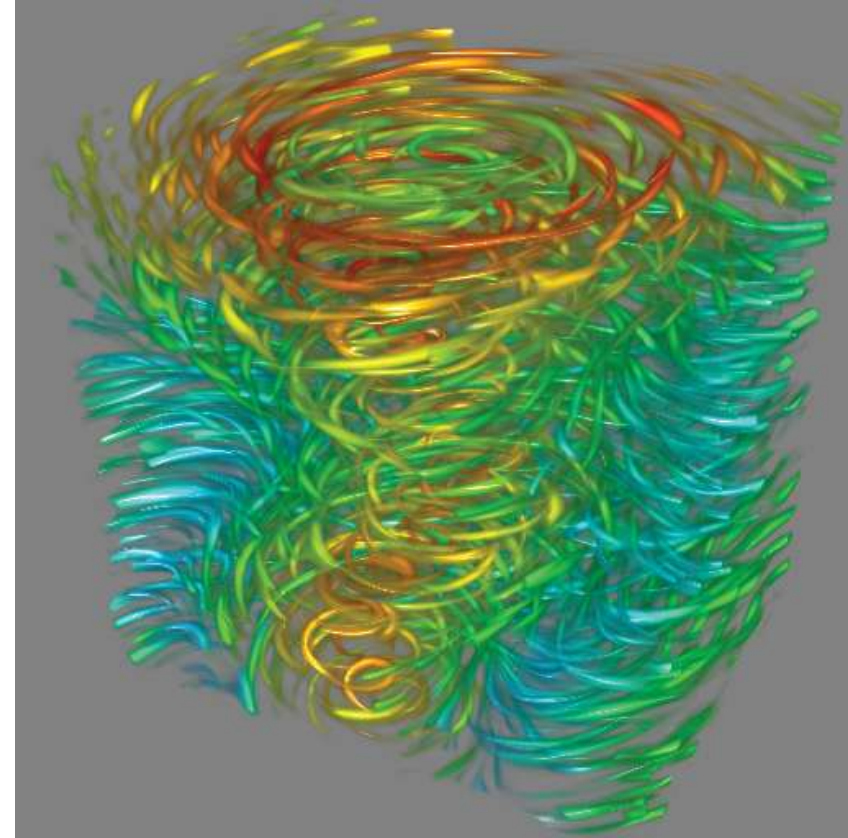


Use scalar quantities such as vorticity magnitude to define opacity for the final rendering.

3D LIC: Density of Noise

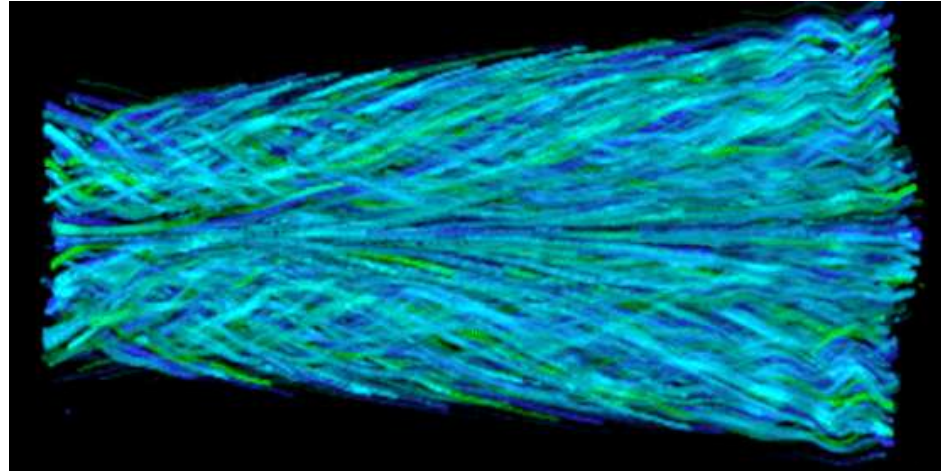


Dense Noise

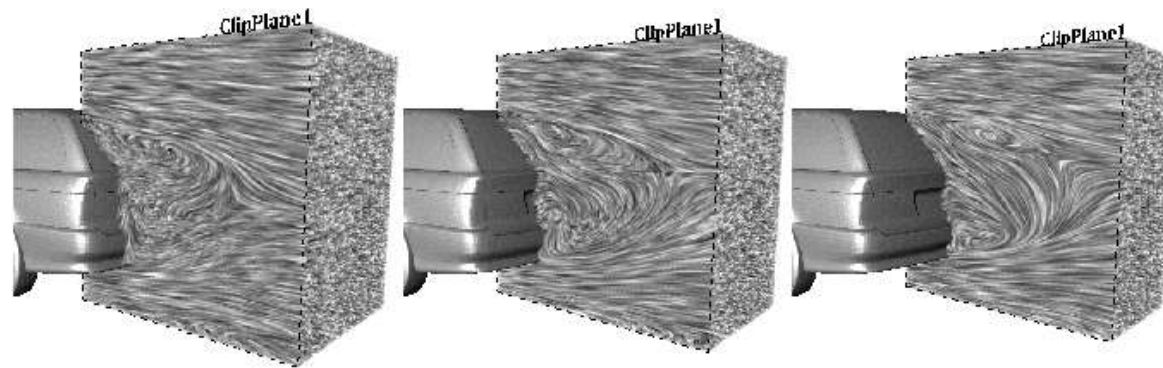


Sparse Noise

3D LIC: Interactive Exploration



3D LIC using volume rendering (from Interrante 97)



3D LIC volume is explored by interactively moving a clip plane
(from Rezk et al 97)

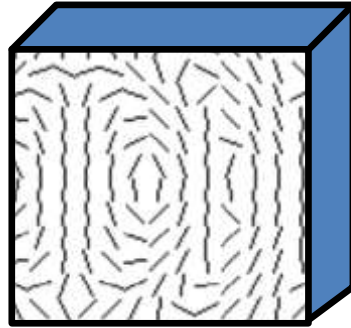
LIC on Surfaces

- Vector fields on or near surfaces can be visualized using LIC textures
- Assumes vector field is everywhere tangential to the surface
 - Natural property of some fields (e.g., lines of curvature)
 - Sometimes meaningful to consider only the tangential part of a full 3D vector field: $\mathbf{v} - (\mathbf{v}\mathbf{n})\mathbf{n}$ (where $\mathbf{n}$ is the surface normal)



Object-Space Surface LIC

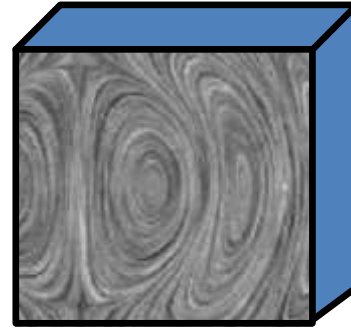
3D object space



Generate



flow texture



Project



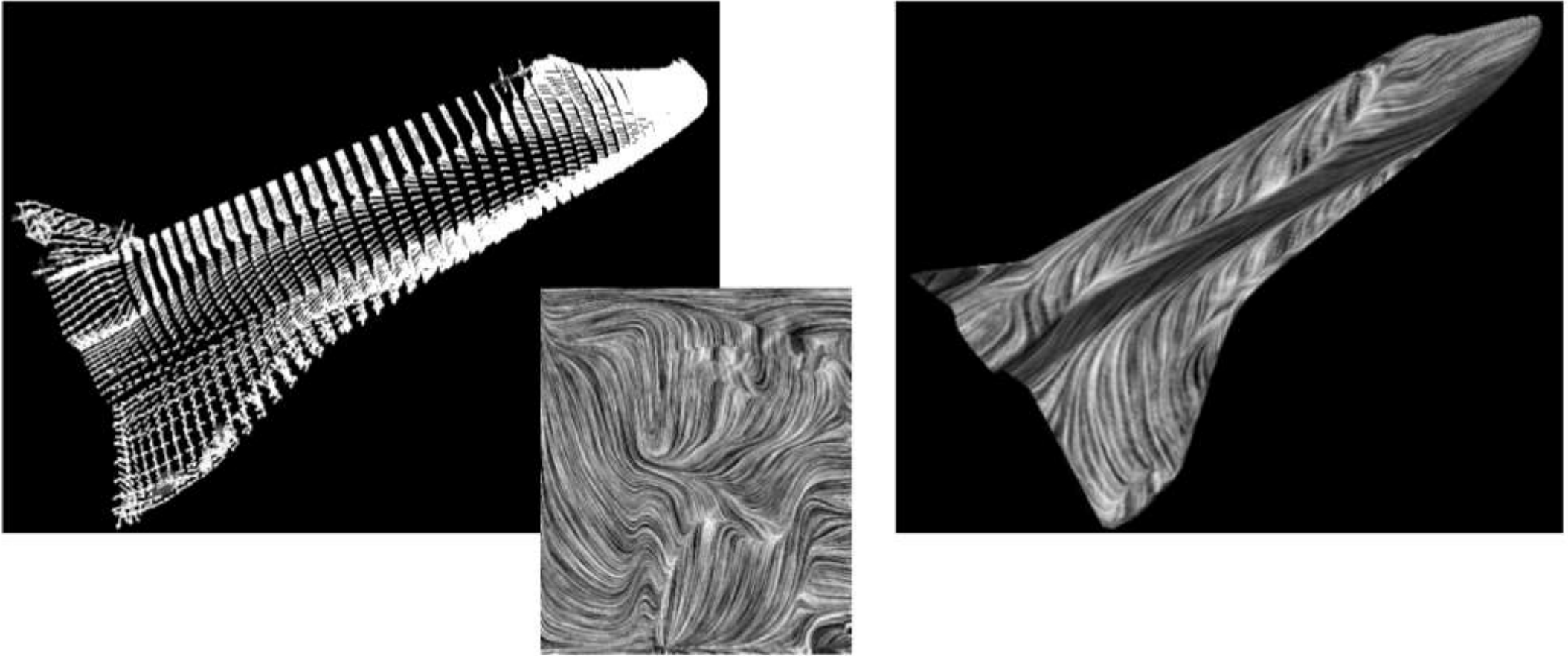
Image plane



Eye

Example: Object-Space Surface LIC

- Flow around a Space Shuttle (global parametrization)



- Problems with global texture: Distortion
 - For ideal results, would have to adapt kernel length and frequency of noise

Example: Local Texture Approach

Approach by [Battke et al. 1996]: Create a separate LIC texture for each triangle

- Does not require a global parametrization
- Does not suffer from distortions
- Integration needs to continue on neighboring triangles

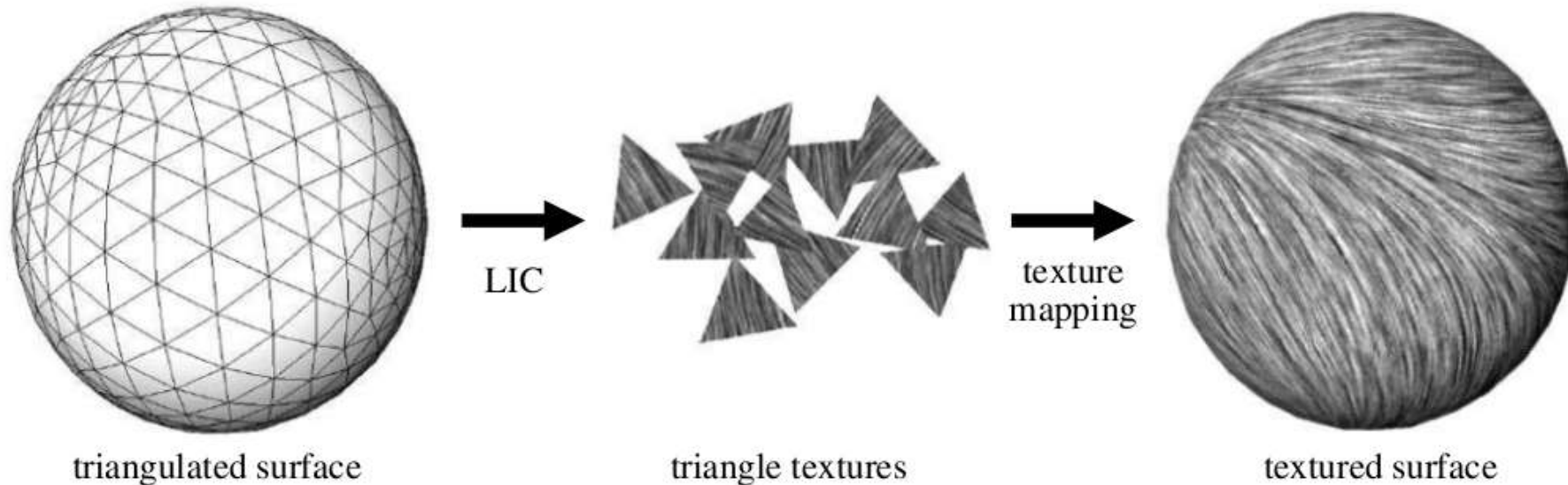
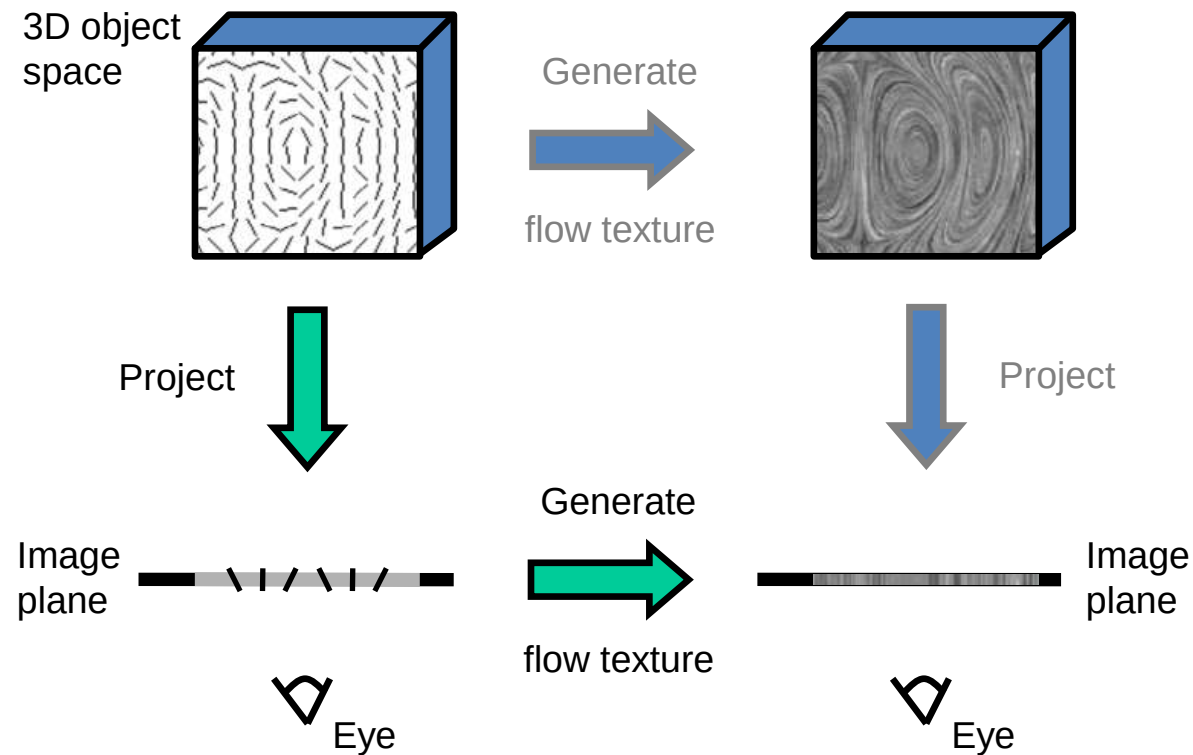


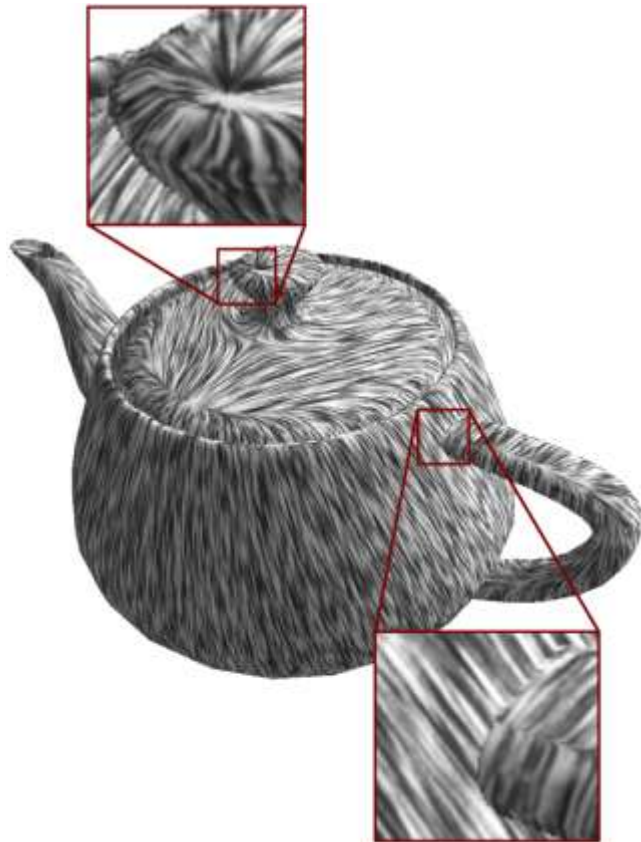
Image-Space Based Surface LIC



Idea: Project shape and vector field onto the image plane first, perform convolution within image space

Image-Space LIC: Discontinuities

- In image-space based surface LIC, extra care has to be taken to avoid convolving across **geometric discontinuities**



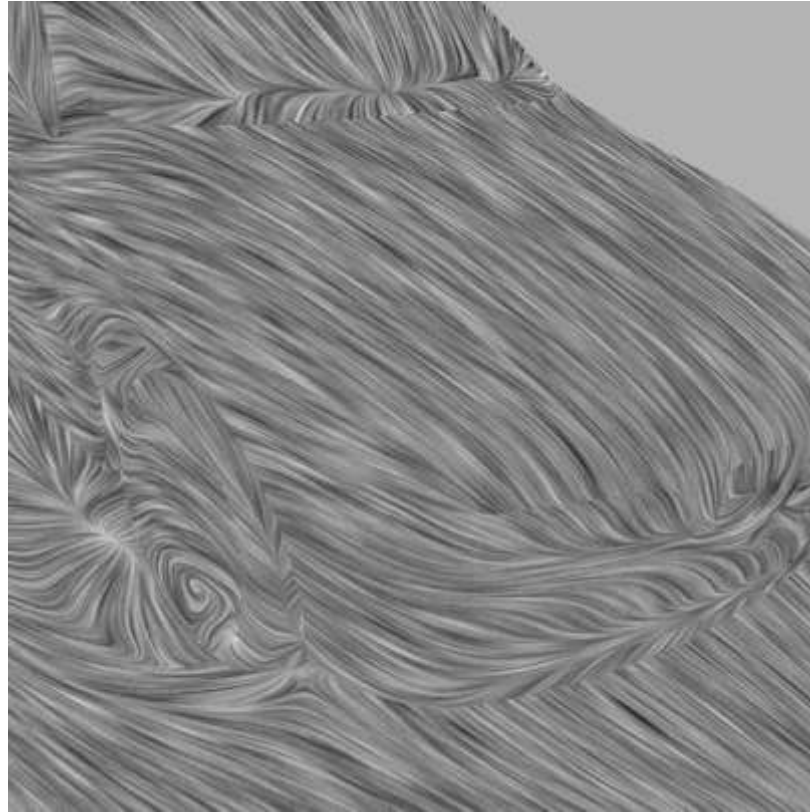
Without edge detection



With edge detection

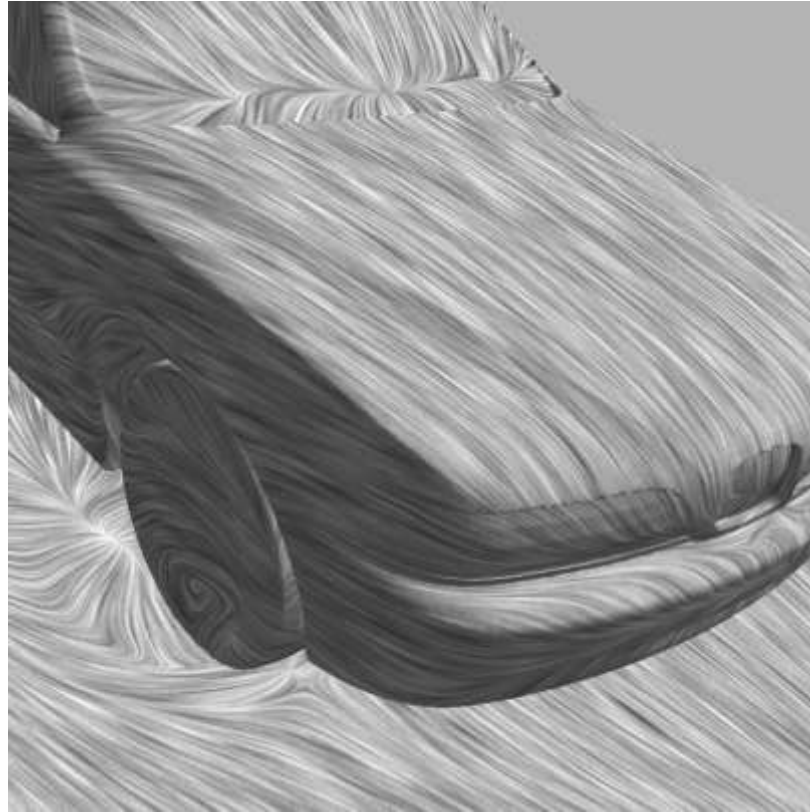
Right image pair from [Laramée et al. 2003]

LIC on Surfaces: No Shading



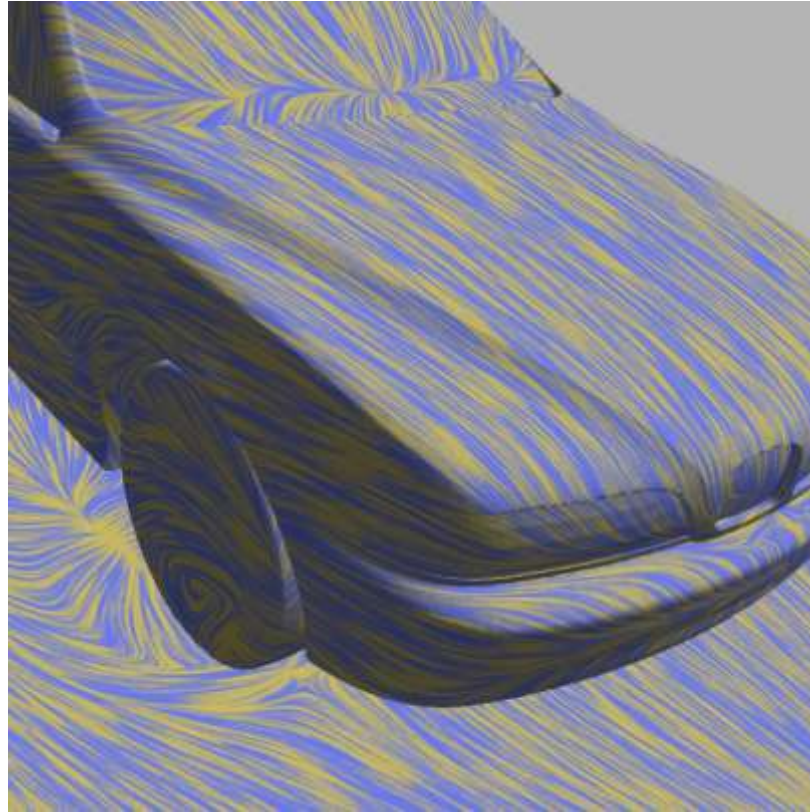
No surface shading

LIC on Surfaces: Gray Shading



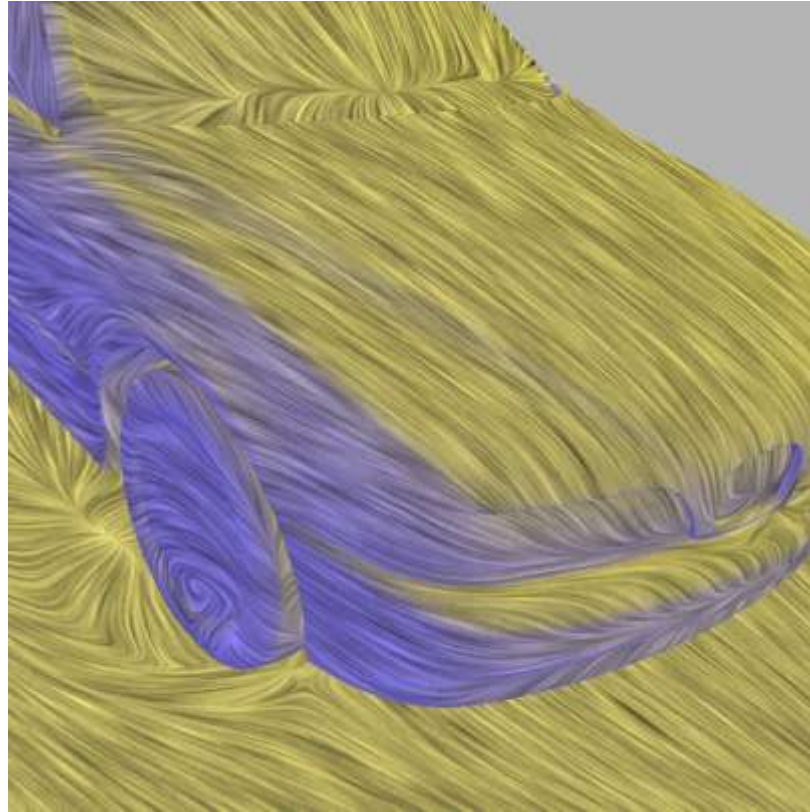
Gray surface

LIC on Surfaces: Yellow/blue texture



Gray surface
Yellow/blue flow

LIC on Surfaces: Cool/warm shading



Cool/warm surface
shading

Summary: 3D LIC

- The main challenge in **3D LIC** is occlusion.
 - Volume rendering with specific transfer functions
 - Sparse noise texture
 - Interactive clipping
- **LIC on surfaces**
 - can be implemented in object space or image space
 - requires careful choice of shading (e.g., cool/warm) to effectively convey LIC texture and surface shape at the same time

Summary: Image- vs. Geometry-based

Geometry-based Flow Vis

- Explicit geometric representation as curve, surface, etc.
- Requires a strategy for seeding
- Some strategies for 3D flow fields (e.g., illuminated stream lines, stream surfaces)

Image-based Flow Vis

- Direct generation of an image / texture
- Usually dense in the domain (possibly sparse input noise)
- Best suited for 2D fields or on surfaces

References: Image-Based Flow Visualization

Survey paper:

- Robert Laramée, Helwig Hauser, Helmut Doleisch, Benjamin Vrolijk, Frits Post, Daniel Weiskopf: The State of the Art in Flow Visualization: Dense and Texture-Based Techniques. Computer Graphics Forum 23(2):203-221, 2004

Individual papers:

- Brian Cabral, Leith Casey Leedom: Imaging vector fields using line integral convolution. ACM SIGGRAPH 1993, pp. 263-270
- D. Stalling and H. Hege. Fast and Resolution Independent Line Integral Convolution. ACM SIGGRAPH 1995, pp. 249–256
- R. Wegenkittl, E. Gröller, W. Purgathofer. Animating Flow Fields: Rendering of Oriented Line Integral Convolution. Proc. Computer Animation 1997, pp. 15–21

Individual Papers (continued)

- T. Urness, V. Interrante, I. Marusic, E. Longmire, B. Ganapathisubramani. Effectively Visualizing Multi-Valued Flow Data using Color and Texture. IEEE Visualization 2003, pp. 115–122
- L. K. Forssell. Visualizing Flow over Curvilinear Grid Surfaces Using Line Integral Convolution. IEEE Visualization 1994, pp. 240–247
- D. Stalling. LIC on Surfaces. In: Texture Synthesis with Line Integral Convolution, ACM SIGGRAPH 1997, pp. 51–64
- C. Rezk-Salama, P. Hastreiter, C. Teitzel and T. Ertl. Interactive exploration of volume line integral convolution based on 3D-texture mapping. IEEE Visualization 1999, pp. 233–240
- V. Interrante, C. Grosch. Strategies for Effectively Visualizing 3D Flow with Volume LIC. IEEE Visualization 1997, pp. 421–424

Exam-like Questions

1. How does the length (support) of the kernel affect the result of line integral convolution (LIC)?
2. What makes FastLIC faster than standard LIC?
3. Suggest a strategy to reduce problems with occlusion when rendering three dimensional LIC.